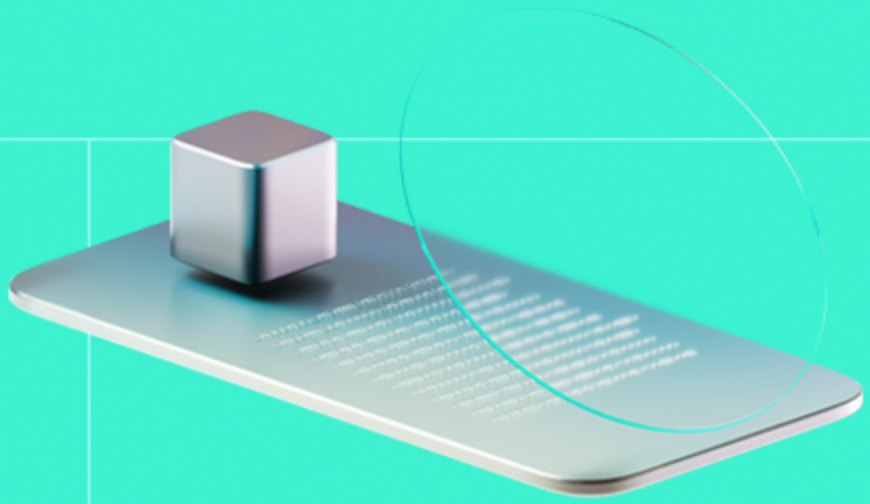
Smart Contract Code Review And Security Analysis Report



Customer: Fabstir

Date: 16/03/2026

We express our gratitude to the Fabstir team for the collaborative engagement that enabled the execution of this Smart Contract Security Assessment.

Fabstir Compute Contracts implements a decentralized peer-to-peer marketplace for AI/LLM inference on EVM-compatible chains (Base, opBNB).

Document

Name	Smart Contract Code Review and Security Analysis Report for Fabstir
Audited By	Seher Saylik; Stepan Chekhovskoi
Approved By	Ivan Bondar
Website	https://fabstir.com
Changelog	18/02/2026 - Preliminary Report
	03/03/2026 - Remediation Report
	10/03/2026 - Second Remediation Report
	16/03/2026 - Final Report
Platform	Base, opBNB
Language	Solidity
Tags	Marketplace; Staking; Upgradable; Signatures; Governance; Voting
Methodology	https://docs.hacken.io/methodologies/smart-contracts

Review Scope

Repository	https://github.com/Fabstir/fabstir-compute-contracts#
Commit	8edc68be530f53249c6845fb0c1fbdf9dec68e62
Retest 1	f614355dbc0fd346dff331a587fc24413a9f0664
Retest 2	df1f2e4d7725a98d76b821ee759cd2c59f9dfa21
Final	fba54b2bf1181dacc7ec8db43e0e16854df0483d

Audit Summary

The system users should acknowledge all the risks summed up in the risks section of the report

41	24	12	5
Total Findings	Resolved	Accepted	Mitigated

Findings by Severity

Severity	Count
Critical	0
High	4
Medium	11
Low	13

Vulnerability	Severity	Status
F-2026-14964 - Proposal Fee Permanently Locked on Rejection	High	Fixed
F-2026-14977 - Token Pricing Assumes All Payment Tokens Have Same Decimals And Price	High	Fixed
F-2026-15254 - Use of IERC20.transfer() Instead of SafeERC20.safeTransfer() in Refund Path	High	Fixed
F-2026-15334 - Delegate Spending Cap Bypass via Token Selection Allows Draining Payer Funds Beyond Intended Limits	High	Fixed
F-2026-14898 - Refund Failure Prevents Host Payment and Locks Session Funds	Medium	Fixed
F-2026-14911 - proofInterval Validated as Token Count but Used as Seconds in Timeout Calculation	Medium	Fixed
F-2026-14966 - User Funds Unexpectedly Locked for Vote	Medium	Accepted
F-2026-14984 - Fee-On-Transfer And Rebasing Tokens Break Accounting And Can Cause Loss Of Funds	Medium	Mitigated
F-2026-15143 - Unregistered or Slashed Hosts Can Continue Operating in Active Sessions	Medium	Accepted
F-2026-15144 - Dispute Window Calculated From Session Start Time Instead of Last Proof Submission	Medium	Fixed
F-2026-15147 - Shared Proof Replay-Prevention State Across Multiple Uncoordinated Entry Points Causes Proof Submission Failure	Medium	Mitigated
F-2026-15149 - Missing Access Control on verifyAndMarkComplete Enables Proof Griefing Attack	Medium	Fixed

Vulnerability	Severity	Status
F-2026-15162 - Deactivated Model Still Usable for Sessions and Node Registration	Medium	Fixed
F-2026-15257 - Early Cancel Fee Applied on Depositor-Triggered Timeouts	Medium	Fixed
F-2026-15280 - Hardcoded Private Key Exposure in Test Scripts Enables Unauthorized Account Control	Medium	Fixed
F-2026-14909 - Proof Verification Bypassed When ProofSystem Is Not Configured	Low	Fixed
F-2026-14913 - Hardcoded Token Rate Limit Cannot Adapt to Future Hardware Improvements	Low	Fixed
F-2026-14917 - Depositors Can Cancel Sessions Immediately - No Protection Window for Hosts	Low	Mitigated
F-2026-14965 - lateVotes Not Cleared on Proposal Execution	Low	Fixed
F-2026-14998 - Signature Malleability Attack in ProofSystemUpgradeable Contract	Low	Fixed
F-2026-15064 - Unfinalized Access-Control Functionality in ProofSystemUpgradeable Contract	Low	Fixed
F-2026-15255 - Delegate Authorization Has No Spending Limits	Low	Fixed
F-2026-15256 - Delegate Scope Is Not Restrictable by Host or Model	Low	Mitigated
F-2026-15258 - setMinTokensFee Function Has No Upper Bound	Low	Accepted
F-2026-15278 - Host Can Unregister During Active Sessions and Withdraw Stake While Continuing to Earn	Low	Accepted
F-2026-15332 - Delegate Cannot Complete or Manage Sessions They Created	Low	Accepted
F-2026-15333 - Delegate Authorization State Can Be Stale and Misleading	Low	Accepted
F-2026-15335 - Delegate Configuration Limited to Single Host/Model — No Multi-Config Support Per Delegate	Low	Accepted
F-2026-14873 - Code Duplication in createSessionFromDeposit()	Info	Accepted
F-2026-14908 - Unused onlyRegisteredHost Modifier Contains Empty Implementation	Info	Fixed
F-2026-14916 - Pre-Deposited Users Cannot Create Model-Specific Sessions	Info	Fixed
F-2026-14967 - Blacklisted Proposer Permanently Blocks Model Execution and Re-Proposals	Info	Accepted
F-2026-14976 - Host Signature In submitProofOfWork() Adds No Security Value	Info	Fixed
F-2026-15002 - Unused Functions In ProofSystemUpgradeable	Info	Fixed
F-2026-15003 - Missing Zero-Address Checks Across Contracts	Info	Accepted
F-2026-15065 - Unfinalized Circuits Functionality in ProofSystemUpgradeable Contract	Info	Fixed
F-2026-15066 - Redundant Function Wrapper in ProofSystemUpgradeable Contract	Info	Fixed

Vulnerability	Severity	Status
F-2026-15067 - Inefficient Revert Messages	Info	Mitigated
F-2026-15279 - verified Field in ProofSubmission Is Hardcoded to true — Dead Storage	Info	Fixed
F-2026-15330 - Redundant usdcAddress State Variable and Setter Introduce Unnecessary Complexity	Info	Accepted
F-2026-15336 - No Function to Remove or Disable an Accepted Token	Info	Accepted

Documentation quality

- Functional requirements are detailed.
  - Project overview is detailed.
  - All roles in the system are described.
  - Use cases are described and detailed.
  - Not all interactions are described.
- Technical description is detailed.
  - Run instructions are provided.
  - The NatSpec documentation is partial.

Code quality

- The development environment is configured.

Test coverage

Code coverage cannot be measured in the current state due to stack limitation errors in the coverage tooling.

- Deployment and basic user interactions are covered with tests.
- Negative cases coverage is partially missed.
- Interactions by several users are not tested thoroughly.

Table of Contents

System Overview **8**

Files In Scope 8

Privileged Roles 9

Potential Risks **10**

Findings **11**

Vulnerability Details 11

Disclaimers 116

Appendix 1. Definitions **117**

Severities 117

Potential Risks 117

Appendix 2. Scope **118**

Appendix 3. Additional Valuables **119**

System Overview

Fabstir Compute Contracts implements a decentralized peer-to-peer marketplace for AI/LLM inference on EVM-compatible chains (Base, opBNB). The protocol connects users who need AI compute (depositors) with GPU node operators (hosts) who run approved AI models, facilitating session-based interactions with on-chain payment settlement.

Hosts register as compute nodes by staking FAB tokens as an economic bond, declaring the AI models they support and their per-token pricing (in both native currency and stablecoins). AI models must first be approved either by the contract owner (trusted tier) or through community governance voting using FAB tokens. Users create sessions by depositing funds (ETH/BNB or ERC20 stablecoins), specifying a host, a model, and pricing terms. During a session, the host submits incremental proofs of work, ECDSA-signed attestations of tokens processed, which are verified on-chain. Conversation data and proofs are stored off-chain via S5 CIDs, with only hashes and references recorded on-chain for cost efficiency.

Upon session completion (voluntary by depositor/host, or forced via timeout), the contract settles payments: the host receives payment proportional to proven work (minus a configurable treasury fee), and the depositor is refunded the unused deposit. Host earnings are accumulated in a dedicated contract to reduce gas costs, allowing batch withdrawals. Misbehaving hosts can be slashed by a designated slashing authority, with slashed FAB tokens sent to the protocol treasury.

All five contracts follow the UUPS (Universal Upgradeable Proxy Standard) pattern using OpenZeppelin's upgradeable libraries, allowing the owner to upgrade implementations while preserving state.

Files in Scope

- **ModelRegistryUpgradeable.sol** - Manages an on-chain registry of approved AI models (identified by HuggingFace repo + filename + SHA256 hash). Implements a two-tier approval system: owner-curated trusted models (tier 1) and community-proposed models approved via FAB token governance voting (tier 2) with anti-sniping vote extensions. Charges a 100 FAB proposal fee and requires 100k FAB approval threshold.
- **NodeRegistryWithModelsUpgradeable.sol** - Manages compute node registration, staking, and model support declarations. Hosts must stake a minimum of 1,000 FAB tokens and declare supported models (verified against ModelRegistry). Supports granular pricing: per-node defaults for native and stable tokens, per-model overrides, and per-token overrides. Includes a slashing mechanism allowing a designated authority to penalize misbehaving hosts (up to 50% per slash with 24-hour cooldown), with automatic unregistration if stake falls below the minimum threshold.
- **JobMarketplaceWithModelsUpgradeable.sol** - The core marketplace contract orchestrating session lifecycle: creation (with native or ERC20 deposits), proof-of-work submission by hosts, session completion/timeout settlement, and treasury fee collection. Supports both direct-deposit session creation and a wallet-agnostic pre-deposit pattern. Enforces host registration, model support, minimum pricing, deposit bounds, and proof rate-limiting. Integrates with HostEarnings for payment accumulation and ProofSystem for signature verification.
- **HostEarningsUpgradeable.sol** - An accumulation contract that batches host earnings from completed sessions to reduce per-job gas costs. Only authorized callers (**JobMarketplace**) can

credit earnings. Hosts can withdraw individual token balances, all balances for a token, or multiple tokens at once. Includes an emergency token rescue function for accidentally sent funds.

- **ProofSystemUpgradeable.sol** - Provides ECDSA signature verification for host proof-of-work submissions. Hosts sign `keccak256(proofHash, prover, claimedTokens)` using `eth_sign`, and the contract recovers the signer to verify authenticity. Tracks verified proof hashes to prevent replay attacks. Supports single and batch (up to 10) proof verification. Also maintains a model-to-circuit registry for future extensibility.

Privileged roles

ModelRegistryUpgradeable.sol:

- **owner**: Can add trusted models (tier 1) and batch-add models, bypassing community governance. Can deactivate and reactivate any model. Can authorize contract upgrades. Can transfer and renounce ownership.

NodeRegistryWithModelsUpgradeable.sol:

- **owner**: Can update the **ModelRegistry** address. Can set the slashing authority and treasury addresses. Can initialize slashing functionality. Can repair corrupt nodes (returning their stake). Can authorize contract upgrades. Can transfer and renounce ownership.
- **slashingAuthority**: Can slash a portion (up to 50%) of any active host's staked FAB tokens, providing evidence and reason. If the slash reduces the host's stake below the minimum threshold (100 FAB), the host is automatically unregistered.

JobMarketplaceWithModelsUpgradeable.sol:

- **owner**: Can set the treasury address and USDC address. Sets `feeBasisPoints` and `disputeWindow` at initialization (no post-initialization setters). Can authorize contract upgrades. Can transfer and renounce ownership.
- **treasuryAddress** (or owner): Can pause and unpause the contract (blocking session creation and proof submission). Can set the proof system address. Can initialize chain configuration. Can add accepted payment tokens and update their min/max deposit bounds.
- **treasuryAddress**: Can withdraw accumulated treasury fees (native and ERC20 tokens).

HostEarningsUpgradeable.sol:

- **owner**: Can authorize or revoke callers that are permitted to credit earnings. Can rescue excess tokens accidentally sent to the contract. Can authorize contract upgrades. Can transfer and renounce ownership.
- **authorizedCallers** (e.g., **JobMarketplace**): Can credit earnings to host balances and send ETH to the contract.

ProofSystemUpgradeable.sol:

- **owner**: Can authorize or revoke callers for `recordVerifiedProof`. Can register model circuits. Can authorize contract upgrades. Can transfer and renounce ownership.
- **authorizedCallers** (or owner): Can record verified proofs (marking proof hashes as used to prevent replay).

Potential Risks

- **Price precision for creating a session:** in `JobMarketplaceWithModelsUpgradeable`, users specifying `pricePerToken` must understand that prices are stored with 1000x precision (PRICE_PRECISION), meaning a value of 1000 equals 1 base unit per token, not 1000 units per token, which could lead to mispricing if not clearly documented in the frontend.
- **Session settlement:** In `JobMarketplaceWithModelsUpgradeable`, if the `HostEarningsUpgradeable` contract is not authorized to receive ETH or if token transfers to it fail (e.g., token blacklist, paused token), the `_settleSessionPayments()` function will revert, permanently locking user deposits until the configuration is fixed or contract is upgraded.
- **Model Selection Trust Assumption:** In `JobMarketplaceWithModelsUpgradeable`, the `modelId` specified during session creation cannot be verified on-chain, meaning users must trust that hosts use the requested model rather than a cheaper alternative.
- **Slashing is fully discretionary:** When a host is slashed, part of their staked FAB (up to 50% per slash, with a 24-hour cooldown) is taken from the contract and sent to the protocol treasury as a penalty. Evidence and reason are only used in events and off-chain review, so the slashing authority can slash any active host within those limits with no on-chain justification, which is a centralization and trust assumption.
- **Unbounded single-voter influence**: In `ModelRegistryUpgradeable.voteOnProposal`, there is no cap on the amount a single address can vote. A whale holding 100k+ FAB can single-handedly meet the `APPROVAL_THRESHOLD` or block any proposal in a single transaction, making community governance effectively plutocratic. Combined with the absence of any quorum requirement on the against-side or minimum voter count, a single well-funded address can unilaterally decide every proposal outcome.
- **Optimistic proof model with no on-chain dispute mechanism:** In **JobMarketplaceWithModelsUpgradeable**, hosts self-sign their own proof submissions and tokensUsed is incremented immediately upon proof acceptance. The `disputeWindow` only delays host-initiated completion, it does not provide any on-chain function for depositors to challenge or reject individual proofs. Settlement always pays based on cumulative `tokensUsed`, so a fraudulent proof that passes the rate limiter is irreversible once submitted. Depositors can only end sessions early via `completeSessionJob`, but this settles including any already-accepted inflated claims. Fraud remediation depends entirely on off-chain slashing, which is reactive and does not compensate affected users.
- A delegate controls session parameters including `pricePerToken`, `maxDuration`, and `proofTimeoutWindow`, enabling a colluding delegate-host pair to set inflated pricing so the host earns the payer's full deposit while delivering minimal work, all within the configured spending cap

Findings

Vulnerability Details

F-2026-14964 - Proposal Fee Permanently Locked on Rejection - High

Description:

In `ModelRegistryUpgradeable` contract, when a user proposes a new model for community approval, they must pay a 100 FAB token proposal fee. This fee is intended to prevent spam proposals. However, the fee is only returned to the proposer if the proposal is approved. When a proposal is rejected, the 100 FAB fee remains permanently locked in the contract with no mechanism to recover it.

The contract lacks:

1. A refund mechanism for rejected proposal fees
2. An owner rescue function for accumulated stuck fees
3. Any alternative use for the collected fees (e.g., treasury transfer, burn)

Over time, this will result in significant FAB tokens becoming permanently inaccessible.

Proposal fee collected on creation:

```
function proposeModel(
    string memory huggingfaceRepo,
    string memory fileName,
    bytes32 sha256Hash
) external nonReentrant {
    // ...

    // Charge proposal fee to prevent spam
    governanceToken.safeTransferFrom(msg.sender, address(this), PROPOSAL_FEE);

    // ...
}
```

Fee only returned on approval:

```
function executeProposal(bytes32 modelId) external {
    ModelProposal storage proposal = proposals[modelId];
    // ...
```

```
    proposal.executed = true;

    bool approved = proposal.votesFor >= APPROVAL_THRESHOLD &&
                    proposal.votesFor > proposal.votesAgainst;
    if (approved) {
        // Add the model
        models[modelId] = proposal.modelData;
        // ...
    }
    // Return proposal fee to proposer if approved
    if (approved) {
        governanceToken.safeTransfer(proposal.proposer, PROPOSAL_FEE);
    }
    // ...
}
```

The contract has no `rescueTokens()` function or any mechanism to withdraw rejected fees. Over the lifetime of the protocol, significant FAB tokens will become permanently inaccessible.

Assets:

- src/ModelRegistryUpgradeable.sol [https://github.com/Fabstir/fabstir-compute-contracts#]

Status: Fixed

Classification

Impact Rate: 3/5

Likelihood Rate: 5/5

Exploitability: Independent

Complexity: Simple

Severity: High

Recommendations

Remediation: Track rejected proposal fees with a dedicated accumulator and add an owner-callable sweep function. In `executeProposal`, when the proposal is not approved, increment the counter instead of silently retaining the fee:

```
uint256 public accumulatedRejectedFees;
// In executeProposal(), replace the existing fee return block:
if (approved) {
    governanceToken.safeTransfer(proposal.proposer, PROPOSAL_FEE);
```

```
} else {
    accumulatedRejectedFees += PROPOSAL_FEE;
}
```

```
function sweepRejectedFees(address recipient) external onlyOwner {
    require(recipient != address(0), "Invalid recipient");
    uint256 amount = accumulatedRejectedFees;
    require(amount > 0, "Nothing to sweep");
    accumulatedRejectedFees = 0;
    governanceToken.safeTransfer(recipient, amount);
}
```

Resolution:

Fixed in `2d7256c`:

ModelRegistryUpgradeable now tracks fees from rejected proposals through `accumulatedRejectedFees`.
During `executeProposal`, approved proposals return `PROPOSAL_FEE` to proposer, while rejected proposals increase `accumulatedRejectedFees` instead of leaving assets unaccounted.

```
if (approved) {
    governanceToken.safeTransfer(proposal.proposer, PROPOSAL_FEE);
} else {
    accumulatedRejectedFees += PROPOSAL_FEE;
}
```

An owner-controlled withdrawal path is present via `withdrawRejectedFees(uint256 amount)`, enabling recovery of accumulated rejected fees and removing permanent lock risk for proposal-fee assets.

```
function withdrawRejectedFees(uint256 amount) external onlyOwner nonReentrant
{
    uint256 toWithdraw = amount == 0 ? accumulatedRejectedFees : amount;
    require(toWithdraw > 0, "No fees to withdraw");
    require(toWithdraw <= accumulatedRejectedFees, "Insufficient accumulated
fees");
    accumulatedRejectedFees -= toWithdraw;
    governanceToken.safeTransfer(msg.sender, toWithdraw);
    emit RejectedFeesWithdrawn(msg.sender, toWithdraw);
}
```

Evidences

PoC

Reproduce:

-Steps to reproduce the issue-

- Deploy `ModelRegistryUpgradeable` via proxy and initialize with a mock FAB token.
- Proposer approves the registry for 100 FAB and calls `proposeModel()`.
- Do not call `voteOnProposal()` at all. Contract balance stays 100 FAB.
- Warp time forward by `PROPOSAL_DURATION` **+ 1** and call `executeProposal()`.
- `executeProposal()` runs; the proposal is rejected (no votes, threshold not met).

Fee is stuck:

- Proposer balance remains 0, and fee is not returned on rejection.
- Contract balance remains 100 FAB.
- There is no function for the proposer or owner to withdraw or recover this fee, so the 100 FAB is permanently locked in the contract.

PoC Code:

```
    function test_ProposalFeeLockedOnRejection() public {
        vm.startPrank(proposer);
        fabToken.approve(address(modelRegistry), PROPOSAL_FEE);
        uint256 proposerBalanceBefore = fabToken.balanceOf(proposer);
        assertEq(proposerBalanceBefore, PROPOSAL_FEE, "Proposer should have 10
0 FAB");
        uint256 contractBalanceBefore = fabToken.balanceOf(address(modelRegist
ry));
        assertEq(contractBalanceBefore, 0, "Contract should have 0 FAB initial
ly");
        // Propose a model
        modelRegistry.proposeModel(
            "SomeOrg/SomeModel",
            "model",
            keccak256("hash")
        );
        vm.stopPrank();
        // Verify fee was transferred to contract
        uint256 proposerBalanceAfterProposal = fabToken.balanceOf(proposer);
        uint256 contractBalanceAfterProposal = fabToken.balanceOf(address(mode
lRegistry));
        assertEq(proposerBalanceAfterProposal, 0, "Proposer should have 0 FAB
after proposal");
        assertEq(contractBalanceAfterProposal, PROPOSAL_FEE, "Contract should
have 100 FAB");
        // No votes for (proposal will be rejected) ============
        bytes32 modelId = modelRegistry.getModelId("SomeOrg/SomeModel", "model
");
        // Do not vote so that contract balance stays at 100 FAB.
```

```
        // Wait for voting period to end
        vm.warp(block.timestamp + PROPOSAL_DURATION + 1);
        modelRegistry.executeProposal(modelId);
        // Verify proposal was rejected
        (,,,,,bool executed,,,) = modelRegistry.proposals(modelId);
        assertTrue(executed, "Proposal should be executed");
        assertFalse(modelRegistry.isModelApproved(modelId), "Model should NOT
be approved");

        //Verify fee is STUCK
        uint256 proposerFinalBalance = fabToken.balanceOf(proposer);
        uint256 contractFinalBalance = fabToken.balanceOf(address(modelRegistr
y));
        // Proposer never gets their fee back
        assertEq(proposerFinalBalance, 0, "Proposer lost their 100 FAB forever
");

        // Fee remains in contract with no way to retrieve
        assertEq(contractFinalBalance, PROPOSAL_FEE, "100 FAB stuck in contrac
t");
    }
```

Results:

```
Ran 1 test for test/POC.sol:ProposalFeeLockedTest
[PASS] test_ProposalFeeLockedOnRejection() (gas: 327861)
Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 826.46µs (222.42
µs CPU time)
```

F-2026-14977 - Token Pricing Assumes All Payment Tokens Have Same Decimals And Price - High

Description:

Pricing logic in the `NodeRegistryWithModelsUpgradeable` contract treats multiple ERC20 payment tokens as if they share same "stable" price and, in practice, one decimals convention. When no token-specific price is set, the same fallback value is used for every accepted token. The contract never adjusts for token decimals or distinct economic value, so minimum-price checks and session economics can be wrong for any token that differs in decimals or price from the "stable" assumption.

```
function getNodePricing(address operator, address token) external view return
s (uint256) {
    if (token == address(0)) {
        return nodes[operator].minPricePerTokenNative;
    } else {
        uint256 customPrice = customTokenPricing[operator][token];
        if (customPrice > 0) {
            return customPrice;
        }
        return nodes[operator].minPricePerTokenStable;  // same for ALL token
s
    }
}
```

For any `token != address(0)` without a custom price, the return value is always `minPricePerTokenStable`. The token parameter is only considered when `customTokenPricing[operator][token] > 0`. Otherwise the protocol assumes all such tokens have the same decimals and the same price.

```
function getModelPricing(address operator, bytes32 modelId, address token) ex
ternal view returns (uint256) {
    // ...
    if (token == address(0)) {
        uint256 modelPrice = modelPricingNative[operator][modelId];
        return modelPrice > 0 ? modelPrice : nodes[operator].minPricePerToken
Native;
    } else {
        uint256 modelPrice = modelPricingStable[operator][modelId];
        return modelPrice > 0 ? modelPrice : nodes[operator].minPricePerToken
Stable;
    }
}
```

For any non-native token, the returned price depends only on “stable” model/node pricing. The token argument is ignored; there is no per-token model pricing and no use of `customTokenPricing`. So for model sessions, all ERC20s are assumed to have the same decimals and price as this single “stable” value.

Impact

- The same raw value is used for every token in the fallback path. Example: `getNodePricing` returns the same number for a 6-decimal and an 18-decimal token:

```
function createSessionJobWithToken(...) external nonReentrant whenNotPaused r
eturns (uint256 jobId) {
    // ...
    uint256 hostMinPrice = nodeRegistry.getNodePricing(host, token);
    require(pricePerToken >= hostMinPrice, "Price below host minimum");
    // ...
}
```

Without custom pricing, every token is treated as having the same value. Users can pay in a cheaper or riskier token while satisfying the single “stable” minimum, or be over-constrained for a stronger collateral token. Correct behavior for multiple tokens requires setting `customTokenPricing` (and/or future per-token model pricing) for every token.

Assets:

- src/JobMarketplaceWithModelsUpgradeable.sol [https://github.com/Fabstir/fabstir-compute-contracts#]
- src/NodeRegistryWithModelsUpgradeable.sol [https://github.com/Fabstir/fabstir-compute-contracts#]

Status: Fixed

Classification

Impact Rate: 4/5

Likelihood Rate: 4/5

Exploitability: Independent

Complexity: Simple

Severity: High

Recommendations

Remediation: Consider avoiding usage of fallback values. Consider reverting if pricing in unsupported token is requested. Require node operator to selectively set price for each of the supported tokens.

Resolution: Fixed in `f614355`:

NodeRegistryWithModelsUpgradeable now enforces explicit per-token model pricing through `modelTokenPricing[operator][modelId][token]`, and `getModelPricing` reverts on missing entries via `require(price > 0, "No model pricing")`.

```
function getModelPricing(address operator, bytes32 modelId, address token) external view returns (uint256) {
    if (nodes[operator].operator == address(0)) return 0;

    uint256 price = modelTokenPricing[operator][modelId][token];
    require(price > 0, "No model pricing");
    return price;
}
```

Shared fallback paths based on a single stable value were removed from active pricing flow, and model session creation now depends on explicit token-specific configuration.

Evidences

Proof of Concept:

Reproduce:

```
    /**
     * @notice PoC: Session can be created with any token at same "price"
     * @dev User pays vastly different economic value depending on token decimals
     */
    function test_POC_SessionCreationIgnoresTokenEconomics() public {
        uint256 pricePerToken = MIN_PRICE_STABLE; // Same for all tokens
        uint256 duration = 1 days;
        uint256 proofInterval = 1000; // MIN_PROVEN_TOKENS = 100, use 1000 to be safe


        // Using larger deposits to pass minimum deposit requirements
        uint256 depositDAI = 100e18;  // 100 DAI
        uint256 depositUSDT = 100e6;  // 100 USDT
```

```
        // Create session with DAI (18 decimals)
        vm.prank(user);
        uint256 sessionIdDAI = marketplace.createSessionJobWithToken(
            host,
            address(dai18Decimals),
            depositDAI,
            pricePerToken,
            duration,
            proofInterval
        );
        assertTrue(sessionIdDAI > 0, "Session created with DAI");


        // Create session with USDT (6 decimals)
        vm.prank(user);
        uint256 sessionIdUSDT = marketplace.createSessionJobWithToken(
            host,
            address(usdt6Decimals),
            depositUSDT,
            pricePerToken,
            duration,
            proofInterval
        );
        assertTrue(sessionIdUSDT > 0, "Session created with USDT");


        // Both sessions were created with same "pricePerToken"
        // Despite DAI having 18 decimals and USDT having 6 decimals
        // The host's minimum price check passes for both with same raw value

        // Economic impact:
        // - DAI deposit: 100e18 = 100 DAI ≈ $100
        // - USDT deposit: 100e6 = 100 USDT ≈ $100
        // But maxTokens calculation uses same formula regardless of decimals
:
        // maxTokens = (deposit * PRICE_PRECISION) / pricePerToken
        // For DAI: (100e18 * 1000) / 1000 = 100e18 tokens (huge!)
        // For USDT: (100e6 * 1000) / 1000 = 100e6 tokens
        // The system treats these very differently despite same economic val
ue
    }
```

Results:

```
[PASS] test_POC_SessionCreationIgnoresTokenEconomics() (gas: 768190)
```

F-2026-15254 - Use of IERC20.transfer() Instead of SafeERC20.safeTransfer() in Refund Path - High

Description:

The `_settleSessionPayments` function uses a raw `IERC20.transfer` call inside a `try` / `catch` block for ERC20 refunds, whereas all other `ERC20` transfer operations across the codebase consistently use **SafeERC20**.`safeTransfer`.

```
try IERC20(session.paymentToken).transfer(session.depositor, userRefund) retu
rns (bool success) {
    if (!success) {
        userDepositsToken[session.depositor][session.paymentToken] += userRef
und;
        emit RefundCreditedToDeposit(jobId, session.depositor, userRefund, se
ssion.paymentToken);
    }
} catch {
    userDepositsToken[session.depositor][session.paymentToken] += userRefund;
    emit RefundCreditedToDeposit(jobId, session.depositor, userRefund, sessio
n.paymentToken);
}
```

Tokens that do not return a value on `transfer()` (e.g. USDT on Ethereum) cause the ABI decoding to fail when the compiler attempts to decode empty returndata into a bool. Per the Solidity specification, "errors during the decoding of the return data inside a try/catch are not caught" — the decoding failure occurs in the caller's execution context, not within the external call, so the catch block is never entered. Instead, the entire transaction reverts, rolling back all state changes including the session status update.

As a result, any session using a no-return token becomes impossible to finalize whenever a non-zero depositor refund exists. `completeSessionJob()` reverts for both the depositor and the host, and `triggerSessionTimeout()` also reverts since it calls the same settlement function. Deposited funds are permanently locked in the contract with no recovery mechanism. Even when the host has submitted legitimate proofs and earned partial payment, the host cannot collect their earnings either, because the settlement reverts before the host payment can be committed.

Found in commit: `f614355`.

Assets:

- src/JobMarketplaceWithModelsUpgradeable.sol [https://github.com/Fabstir/fabstir-compute-contracts#]

Status: Fixed

Classification

Impact Rate: 5/5

Likelihood Rate: 4/5

Exploitability: Independent

Complexity: Simple

Severity: High

Recommendations

Remediation: Replace the raw `IERC20.transfer()` call with `SafeERC20.safeTransfer()` inside the try/catch block. `safeTransfer()` handles tokens with no return value by treating empty returndata as success, while still reverting on actual transfer failures. This ensures the catch block is only entered when the transfer genuinely fails, making the fallback credit to `userDepositsToken` safe and correct.

Resolution: The raw `IERC20.transfer` call has been replaced with a low-level call that correctly handles tokens with no return value by treating empty returndata as success, while falling back to crediting the depositor's internal balance when the transfer fails for any reason:

```
// F202615254: Low-level call handles non-returning tokens (USDT)
(bool callOk, bytes memory ret) = session.paymentToken.call(
    abi.encodeCall(IERC20.transfer, (session.depositor, userRefund))
);
if (!(callOk && (ret.length == 0 || abi.decode(ret, (bool))))) {
     userDepositsToken[session.depositor][session.paymentToken] += userRefund
;
     emit RefundCreditedToDeposit(jobId, session.depositor, userRefund, sessi
on.paymentToken);
}
```

Revised commit: `df1f2e4`.

Evidences

PoC

Reproduce:

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.24;

import "forge-std/Test.sol";
import "../src/JobMarketplaceWithModelsUpgradeable.sol";
import "@openzeppelin/contracts/proxy/ERC1967/ERC1967Proxy.sol";

/// @dev Token whose transfer() and transferFrom() succeed but return nothing
,
contract NoReturnToken {
    mapping(address => uint256) public balances;
    mapping(address => mapping(address => uint256)) public allowances;

    function balanceOf(address account) external view returns (uint256) {
        return balances[account];
    }

    function approve(address spender, uint256 amount) external returns (bool)
{
        allowances[msg.sender][spender] = amount;
        return true;
    }

    function transferFrom(address from, address to, uint256 amount) external
{
        require(balances[from] >= amount, "insufficient");
        if (from != msg.sender) {
            require(allowances[from][msg.sender] >= amount, "no allowance");
            allowances[from][msg.sender] -= amount;
        }
        balances[from] -= amount;
        balances[to] += amount;
    }

    function transfer(address to, uint256 amount) external {
        require(balances[msg.sender] >= amount, "insufficient");
        balances[msg.sender] -= amount;
        balances[to] += amount;
    }

    function mint(address to, uint256 amount) external {
        balances[to] += amount;
    }
}

contract NoReturnTokenRefundDoS is Test {
    JobMarketplaceWithModelsUpgradeable marketplace;
    NoReturnToken token;
```

```
    address nodeRegistry = makeAddr("nodeRegistry");
    address modelReg     = makeAddr("modelRegistry");
    address hostEarn     = makeAddr("hostEarnings");
    address proofSys     = makeAddr("proofSystem");
    address host         = makeAddr("host");
    address depositor    = makeAddr("depositor");

    bytes32 modelId = keccak256("testModel");

    uint256 constant DEPOSIT   = 1_000_000; // 1 USDT (6 decimals)
    uint256 constant PRICE     = 100;
    uint256 constant INTERVAL  = 1000;
    uint256 constant DURATION  = 1 hours;
    uint256 constant TIMEOUT   = 300;

    function setUp() public {
        //..
    }

    function test_NoReturnToken_PermanentFundLock() public {
        vm.startPrank(depositor);
        token.approve(address(marketplace), DEPOSIT);

        uint256 jobId = marketplace.createSessionJobForModelWithToken(
            host, modelId, address(token), DEPOSIT, PRICE, DURATION, INTERVAL
, TIMEOUT
        );
        vm.stopPrank();

        assertEq(token.balances(address(marketplace)), DEPOSIT, "marketplace
holds deposit");
        assertEq(token.balances(depositor), 0, "depositor has 0");


        // Solidity can't decode empty returndata as bool -  revert and not c
aught by catch.
        vm.prank(depositor);
        vm.expectRevert();
        marketplace.completeSessionJob(jobId, "");

        //Host tries to complete after dispute window  also reverts
        vm.warp(block.timestamp + 31);
        vm.prank(host);
        vm.expectRevert();
        marketplace.completeSessionJob(jobId, "");


        //Funds are permanently locked
```

```
        assertEq(token.balances(address(marketplace)), DEPOSIT, "deposit stuc
k in contract forever");
        assertEq(token.balances(depositor), 0, "depositor cannot recover fund
s");
    }

}
```

F-2026-15334 - Delegate Spending Cap Bypass via Token Selection Allows Draining Payer Funds Beyond Intended Limits - High

Description:

The `DelegateConfig` struct provides spending controls (`maxPerSession`, `totalCap`, `spent`) to protect payers against delegate overspending or key compromise. However, these controls compare raw token amounts without any reference to which token is being used. The struct has no `allowedToken` field, and the delegate freely selects the payment token at session creation:

```
    function createSessionForModelAsDelegate(
        address payer,
        bytes32 modelId,
        address host,
        address paymentToken,
        uint256 amount,
        uint256 pricePerToken,
        uint256 maxDuration,
        uint256 proofInterval,
        uint256 proofTimeoutWindow
    ) external nonReentrant whenNotPaused returns (uint256 sessionId) {
        require(payer != address(0), "No payer");
        if (msg.sender != payer) {
            DelegateConfig storage dc = delegateConfigs[payer][msg.sender];
            require(dc.active, "Not delegate");
            if (dc.validUntil > 0) require(block.timestamp <= dc.validUntil, "
Expired");
            if (dc.maxPerSession > 0) require(amount <= dc.maxPerSession, "Ove
r limit");
            if (dc.totalCap > 0) require(dc.spent + amount <= dc.totalCap, "Ov
er cap");
            if (dc.allowedHost != address(0)) require(host == dc.allowedHost,
"Wrong host");
            if (dc.allowedModel != bytes32(0)) require(modelId == dc.allowedMo
del, "Wrong model");
            require(amount <= type(uint128).max, "Overflow");
            dc.spent += uint128(amount);
        }
// … rest of the code
}
```

Since accepted tokens can have different decimals (e.g., USDC with 6, DAI with 18), the same raw `totalCap` value represents vastly different token quantities depending on which token the delegate chooses. The

cumulative spent counter compounds the problem by summing raw amounts across sessions that may use entirely different tokens.

The spending cap is the primary safeguard protecting payers when delegating session creation to third-party accounts. A malicious delegate can bypass the intended limit by selecting a token with fewer decimals than what the payer calibrated the cap for, pulling funds up to the payer's token approval and `tokenMaxDeposits` ceiling rather than the intended cap. The spent counter also becomes a meaningless accounting record since it sums raw amounts of tokens with different decimals.

Found in commit : `df1f2e4`.

Assets:

- src/JobMarketplaceWithModelsUpgradeable.sol [https://github.com/Fabstir/fabstir-compute-contracts#]

Status: Fixed

Classification

Impact Rate: 4/5

Likelihood Rate: 5/5

Exploitability: Independent

Complexity: Simple

Severity: High

Recommendations

Remediation: Restructure the delegate config mapping to be per-token, so that spending controls are always denominated in the same token they apply to:

```
// payer => delegate => token => DelegateConfig
mapping(address => mapping(address => mapping(address => DelegateConfig))) pu
blic delegateConfigs;
```

Update `configureDelegate()` to accept a token parameter, allowing payers to set separate limits for each token:

```
function configureDelegate(
    address delegate,
    address token,
    uint128 maxPerSession,
    uint128 totalCap,
    uint64 validUntil,
```

```
    address[] allowedHost,
    bytes32[] allowedModel
) external { ... }
```

Resolution: An `allowedToken` field has been added to `DelegateConfig` and is enforced in `createSessionForModelAsDelegate()`, allowing payers to restrict delegates to a specific payment token and ensuring spending caps are denominated in a single token's units. However, it should be noted that this approach only allows configuring a delegate for a single token, host, and model per delegate address — payers who need a delegate to operate across multiple tokens, hosts, or models must either grant unrestricted access or manage separate delegate addresses for each combination.

```
    function createSessionForModelAsDelegate(...) external nonReentrant whenN
otPaused returns (uint256 sessionId) {
        require(payer != address(0), "No payer");
        if (msg.sender != payer) {
            DelegateConfig storage dc = delegateConfigs[payer][msg.sender];
            require(dc.active, "Not delegate");
            if (dc.validUntil > 0) require(block.timestamp <= dc.validUntil, "
Expired");
            if (dc.maxPerSession > 0) require(amount <= dc.maxPerSession, "Ove
r limit");
            if (dc.totalCap > 0) require(dc.spent + amount <= dc.totalCap, "Ov
er cap");
            if (dc.allowedHost != address(0)) require(host == dc.allowedHost,
"Wrong host");
            if (dc.allowedModel != bytes32(0)) require(modelId == dc.allowedMo
del, "Wrong model");
            if (dc.allowedToken != address(0)) require(paymentToken == dc.allo
wedToken, "Wrong token");
            require(amount <= type(uint128).max, "Overflow");
            dc.spent += uint128(amount);
        }
    // ... rest of the code
    }
```

Revised commit: `fba54b2`.

F-2026-14898 - Refund Failure Prevents Host Payment and Locks Session Funds - Medium

Description:

The `_settleSessionPayments()` internal function processes both host payment and user refund within a single atomic transaction. When the user refund transfer fails, the entire transaction reverts, causing the host payment to also be cancelled even though the host has legitimately earned their tokens. The function executes in the following order:

1. Calculate `hostPayment` and `userRefund` amounts
2. Transfer `hostPayment` to `HostEarningsUpgradeable` contract
3. Transfer `userRefund` to `session.depositor`

If step 3 fails, all state changes including step 2 are reverted. The refund transfer may fail due to:

- Depositor being a smart contract that rejects ETH transfers
- Depositor address being blacklisted by the payment token (e.g., USDC, USDT)

When this occurs, both `completeSessionJob()` and `triggerSessionTimeout()` will permanently revert, leaving the session in **Active** status indefinitely with all funds locked in the contract.

```
function _settleSessionPayments(uint256 jobId, address completedBy) internal
{
   SessionJob storage session = sessionJobs[jobId];
   uint256 hostPayment = (session.tokensUsed * session.pricePerToken) / PRICE
_PRECISION;
   uint256 userRefund = session.deposit > hostPayment ? session.deposit - hos
tPayment : 0;
   if (hostPayment > 0) {
       // Host payment logic - succeeds but reverts if refund fails
       // ...
   }
   if (userRefund > 0) {
       if (session.paymentToken == address(0)) {
           (bool sent,) = payable(session.depositor).call{value: userRefund}(
"");
           require(sent, "ETH refund failed"); //Reverts entire transaction
       } else {
           IERC20(session.paymentToken).safeTransfer(session.depositor, userR
efund); //Reverts entire transaction
       }
       session.refundedToUser = userRefund;
```

```
    }
}
```

Hosts who have completed legitimate work and submitted valid proofs cannot receive their earned payments

Assets:

- src/JobMarketplaceWithModelsUpgradeable.sol [https://github.com/Fabstir/fabstir-compute-contracts#]

Status: Fixed

Classification

Impact Rate: 4/5

Likelihood Rate: 3/5

Exploitability: Independent

Complexity: Simple

Severity: Medium

Recommendations

Remediation: On refund failure, credit the refund to the depositor's pre-deposit balance within **JobMarketplace** rather than reverting. The user can then withdraw via the existing `withdrawNative` / `withdrawToken` functions:

```
if (userRefund > 0) {
    if (session.paymentToken == address(0)) {
        (bool sent,) = payable(session.depositor).call{value: userRefund}("")
;
        if (!sent) {
            userDepositsNative[session.depositor] += userRefund;
        }
    } else {
        try IERC20(session.paymentToken).safeTransfer(session.depositor, user
Refund) {
        } catch {
            userDepositsToken[session.depositor][session.paymentToken] += use
rRefund;
        }
    }
    session.refundedToUser = userRefund;
}
```

This reuses existing infrastructure, keeps user and host accounting separate, and ensures settlement never reverts due to a refund failure. The depositor can retrieve funds via the standard withdrawal path at their convenience.

Resolution: Fixed in `8a7331a`:

Settlement logic now uses a pull-pattern fallback for refund failures instead of reverting the full transaction.

On native refund failure, the amount is credited to `userDepositsNative[depositor];` on ERC20 refund failure, the amount is credited to `userDepositsToken[depositor][token]`, with `RefundCreditedToDeposit` emitted in both fallback cases.

```
if (userRefund > 0) {
    session.refundedToUser = userRefund;
    if (session.paymentToken == address(0)) {
        (bool sent,) = payable(session.depositor).call{value: userRefund}("")
;
        if (!sent) {
            // F202614898: Credit to deposit balance on ETH refund failure
            userDepositsNative[session.depositor] += userRefund;
            emit RefundCreditedToDeposit(jobId, session.depositor, userRefund
, address(0));
        }
    } else {
        // F202614898: Use try/catch with low-level transfer for pull pattern
        try IERC20(session.paymentToken).transfer(session.depositor, userRefu
nd) returns (bool success) {
            if (!success) {
                userDepositsToken[session.depositor][session.paymentToken] +=
userRefund;
                emit RefundCreditedToDeposit(jobId, session.depositor, userRe
fund, session.paymentToken);
            }
        } catch {
            userDepositsToken[session.depositor][session.paymentToken] += use
rRefund;
            emit RefundCreditedToDeposit(jobId, session.depositor, userRefund
, session.paymentToken);
        }
}
```

Host settlement and treasury accounting therefore proceed even when direct refund transfer fails, removing the prior all-or-nothing failure mode that could block earned host payments.

F-2026-14911 - proofInterval Validated as Token Count but Used as Seconds in Timeout Calculation - Medium

Description:

The `proofInterval` parameter has conflicting interpretations in the contract. During session creation, it is validated as a token count with a minimum of 100 tokens (`MIN_PROVEN_TOKENS`). However, in `triggerSessionTimeout()`, it is used as a time duration in seconds to determine when a session can be timed out.This causes unpredictable timeout behavior where the timeout window is directly tied to the token count rather than an intended time period.

```
    function _validateProofRequirements(uint256 proofInterval, uint256 deposi
t, uint256 pricePerToken) internal pure {
        // With PRICE_PRECISION: maxTokens = deposit * PRICE_PRECISION / price
PerToken
        uint256 maxTokens = (deposit * PRICE_PRECISION) / pricePerToken;
        uint256 tokensPerProof = proofInterval;
        require(tokensPerProof >= MIN_PROVEN_TOKENS, "Proof interval too small
");
        require(maxTokens >= tokensPerProof, "Deposit too small for proof inte
rval");
    }
```

```
    function triggerSessionTimeout(uint256 jobId) external nonReentrant {
        // ...
        bool hasTimedOut = (block.timestamp > session.startTime + session.maxD
uration)
            || (block.timestamp > session.lastProofTime + session.proofInterva
l * 3);
        require(hasTimedOut, "Session not timed out");
        // ,,,
    }
```

- Users cannot independently control token-per-proof requirements and timeout behavior
- Sessions may timeout unexpectedly early or remain active much longer than intended

Assets:

- src/JobMarketplaceWithModelsUpgradeable.sol [https://github.com/Fabstir/fabstir-compute-contracts#]

Status: Fixed

Classification

Impact Rate: 3/5

Likelihood Rate: 4/5

Exploitability: Independent

Complexity: Simple

Severity: Medium

Recommendations

Remediation: Add a separate `minTokensPerProof` field for time-based timeout logic, keeping `proofInterval` for detecting abandoned sessions validation only:

```
struct SessionJob {
    // ...
    uint256 minTokensPerProof;      // Minimum tokens host must claim per pro
of
    uint256 proofTimeoutSeconds;    // Time in seconds before session conside
red abandoned
    // ...
}
```

In `_validateProofRequirements()`:

```
function _validateProofRequirements(
   uint256 minTokensPerProof,
   uint256 deposit,
   uint256 pricePerToken
) internal pure {
   uint256 maxTokens = (deposit * PRICE_PRECISION) / pricePerToken;
   require(minTokensPerProof >= MIN_PROVEN_TOKENS, "Tokens per proof too smal
l");
   require(maxTokens >= minTokensPerProof, "Deposit too small for proof requi
rements");
}
```

Resolution: Fixed in `8893036`:

Timeout semantics were separated from token-per-proof requirements by introducing `proofTimeoutWindow` as an explicit time-based parameter in session configuration and storage.

```
uint256 proofTimeoutWindow;
```

Validation now enforces bounded timeout values (`MIN_PROOF_TIMEOUT` to `MAX_PROOF_TIMEOUT`), while proof token requirements remain validated through proofInterval and `MIN_PROVEN_TOKENS`.

```
 require(
            params.proofTimeoutWindow >= MIN_PROOF_TIMEOUT && params.proofTime
outWindow <= MAX_PROOF_TIMEOUT,
            "Invalid proof timeout window"
        );
```

`triggerSessionTimeout` now evaluates abandonment using `lastProofTime + proofTimeoutWindow` (with `DEFAULT_PROOF_TIMEOUT` fallback for legacy sessions), eliminating the prior coupling of timeout behavior to token-count configuration.

```
 uint256 timeoutWindow = session.proofTimeoutWindow > 0
     ? session.proofTimeoutWindow
     : DEFAULT_PROOF_TIMEOUT;
     || (block.timestamp > session.lastProofTime + timeoutWindow);
```

F-2026-14966 - User Funds Unexpectedly Locked for Vote - Medium

Description:

The `withdrawVotes` function of the `ModelRegistryUpgradeable` contract checks withdrawal eligibility against the current proposals state. Either proposal should be executed (model accepted or rejected) or 7 days should pass (in case proposal was not executed).

The contract allows certain model to be re-proposed in case of failure. User may not retrieve funds after unsuccessful proposal. In case model is re-proposed, users would not be able to withdraw funds locked during the previous iteration, however, those funds are not considered as votes for / against in the new proposal as the new proposal starts with clear state.

```
function withdrawVotes(bytes32 modelId) external nonReentrant {
  ModelProposal storage proposal = proposals[modelId];
  require(proposal.executed ||
          block.timestamp > proposal.endTime + 7 days,
          "Cannot withdraw yet");
  uint256 amount = votes[modelId][msg.sender];
  require(amount > 0, "No votes to withdraw");
  votes[modelId][msg.sender] = 0;
  governanceToken.safeTransfer(msg.sender, amount);
}
```

This may lead to additional unexpected locks of user funds for additional period without any benefit for the new proposal.

Users must actively monitor for re-proposals and withdraw during the narrow cooldown window to avoid extended lockup.

Assets:

- src/ModelRegistryUpgradeable.sol [https://github.com/Fabstir/fabstir-compute-contracts#]

Status: Accepted

Classification

Impact Rate: 3/5

Likelihood Rate: 4/5

Exploitability: Independent

Complexity: Simple

Severity: Medium

Recommendations

Remediation: Consider refactoring voting system to distinguish old proposals from the new ones and process votes withdrawals smoothly.

Resolution: Accepted:

The issue is acknowledged as an intentional governance behavior. Vote assets remain locked for the defined voting period to prevent double-voting and last-minute liquidity manipulation. The lock duration is bounded by protocol constants and aligned with common governance escrow patterns.

The client-provided comment:

> By design. Funds are locked during the voting period to prevent double-voting and last-minute liquidity attacks. The locking window is bounded by the vote duration constant. This is consistent with standard governance escrow patterns (e.g., OpenZeppelin Governor).

F-2026-14984 - Fee-On-Transfer And Rebasing Tokens Break Accounting And Can Cause Loss Of Funds - Medium

Description:

Fee-on-transfer tokens take a fee on each `transfer` / `transferFrom`, so the recipient gets less than the sent amount. Rebasing tokens change holder balances (e.g. rewards) without a matching transfer. In both cases, the contract's actual balance change can differ from the amount passed in the call.

The codebase does not measure actual received amounts (e.g. via `balanceOf` before/after). It always credits and later pays out the nominal amount. So total internal accounting can exceed the contract's real token balance. When paying out, the contract tries to send more than it holds, so some transfers revert, funds get stuck, and settlements can be wrong or unfair depending on withdrawal order.

1. `NodeRegistryWithModelsUpgradeable`
   - `registerNode`: `safeTransferFrom()` for deposit.
   - `stake`: `safeTransferFrom()` for deposit.
   - `unregisterNode` / `slashStake` / `repairCorruptNode`: Outgoing transfers use `stakedAmount` or derived amounts. Total accounting may exceed actual FAB balance.
2. `JobMarketplaceWithModelsUpgradeable`
   - `createSessionJobWithToken` / `createSessionJobForModelWithToken`: `safeTransferFrom()` for deposit; `session.deposit` set to nominal amount.
   - `_settleSessionPayments`: Pays `host` and `depositor` from `session.deposit` / `tokensUsed`. Outgoing `safeTransfer()` may exceed real balance if payment token is fee-on-transfer.
3. `JobMarketplaceWithModelsUpgradeable`
   - `depositToken`: `safeTransferFrom()` for deposit.
   - `withdrawToken`: Sends amount from contract. Sum of user balances may exceed contract balance in case of non-standard token.
4. `ModelRegistryUpgradeable`
   - `proposeModel`: `safeTransferFrom()` for proposal fee deposit.
   - `voteOnProposal`: `safeTransferFrom()` for user vote deposit.
   - `withdrawVotes`: `safeTransfer` for locked votes withdrawal. Sum of user balances may exceed contract balance in case of non-standard token.
5. `HostEarningsUpgradeable`

- `creditEarnings`: Receives `netHostPayment` from `JobMarketplace`; credits that nominal amount.
- `JobMarketplace`: Does `safeTransfer`(`hostEarnings`, `netHostPayment`). If token is fee-on-transfer, contract receives less but still credits full amount → host withdrawals can revert or over-draw.

Assets:

- src/JobMarketplaceWithModelsUpgradeable.sol [https://github.com/Fabstir/fabstir-compute-contracts#]
- src/NodeRegistryWithModelsUpgradeable.sol [https://github.com/Fabstir/fabstir-compute-contracts#]
- src/ModelRegistryUpgradeable.sol [https://github.com/Fabstir/fabstir-compute-contracts#]
- src/HostEarningsUpgradeable.sol [https://github.com/Fabstir/fabstir-compute-contracts#]

Status: Mitigated

Classification

Impact Rate: 3/5

Likelihood Rate: 3/5

Exploitability: Independent

Complexity: Simple

Severity: Medium

Recommendations

Remediation: Either explicitly disallow fee-on-transfer and rebasing tokens when configuring accepted tokens and staking/governance tokens (preferred for simplicity), or make accounting use the actually received amount.

If fee-on-transfer tokens are supported, compute `received = balanceOf(address(this))` after the transfer minus balance before, and use received (not the nominal amount) for all internal balances — staked amounts, session deposits, vote balances, treasury and earnings credits — so that internal accounting always matches the contract's real token balance.

If rebasing tokens are supported, consider storing user balances as a shared pool of shares. wrapping incoming funds to shares and unwrapping shares to funds on withdrawal.

Resolution: Mitigated:

Based on client-provided confirmation, fee-on-transfer and rebasing tokens will not be supported. Token onboarding is restricted to an owner-controlled allowlist, and accepted assets are screened before activation. This operational constraint reduces practical exposure despite nominal-amount transfer accounting remaining in code.

The client-provided comment:

> Mitigated by strict token whitelist. The marketplace accepts only owner-approved tokens via setAcceptedToken(). The only accepted ERC-20 is USDC on Base, which is neither fee-on-transfer nor rebasing. Future token additions will be screened before acceptance

F-2026-15143 - Unregistered or Slashed Hosts Can Continue Operating in Active Sessions - Medium

Description:

The `submitProofOfWork` and `completeSessionJob` functions in **JobMarketplaceWithModelsUpgradeable** do not verify whether the host is still registered and active in **NodeRegistryWithModelsUpgradeable**. This allows hosts who have been unregistered or slashed for misbehavior to continue submitting proofs and receiving payments from existing sessions.

Host registration status is only validated at session creation time, not during proof submission or session completion.

```
// src/JobMarketplaceWithModelsUpgradeable.sol:538
_validateHostRegistration(params.host);
// src/JobMarketplaceWithModelsUpgradeable.sol:487-503
function _validateHostRegistration(address host) internal view {
   require(host != address(0), "Invalid host address");

   (address operator, , bool active, , , , , ) = nodeRegistry.getNodeFullInfo
(host);

   require(operator != address(0), "Host not registered");
   require(active, "Host not active");  // Checked here
}
```

However, `submitProofOfWork` only checks if the caller matches the stored host address:

```
// src/JobMarketplaceWithModelsUpgradeable.sol:585-587
function submitProofOfWork(...) external nonReentrant whenNotPaused {
   SessionJob storage session = sessionJobs[jobId];
   require(session.status == SessionStatus.Active, "Session not active");
   require(msg.sender == session.host, "Only host can submit proof");
   // No check if host is still registered/active in NodeRegistry
   // ...
}
```

Similarly, `completeSessionJob` lacks this validation:

```
// src/JobMarketplaceWithModelsUpgradeable.sol:662-670
function completeSessionJob(uint256 jobId, string calldata conversationCID) e
xternal nonReentrant {
   SessionJob storage session = sessionJobs[jobId];
   require(session.status == SessionStatus.Active, "Session not active");
```

```
    require(
        msg.sender == session.depositor || msg.sender == session.host,
        "Only depositor or host can complete"
    );
    // No check if host is still registered/active
    // ...
}
```

When a host unregisters or is slashed, their data is deleted from NodeRegistry:

```
// src/NodeRegistryWithModelsUpgradeable.sol:393
delete nodes[msg.sender];  // All host data removed
// src/NodeRegistryWithModelsUpgradeable.sol:627 (after slash auto-unregister
)
delete nodes[host];
```

Attack Scenario 1: Slashed Host Continues Earning

1. User creates session with `Host_A` (deposit: 100 USDC)
2. `Host_A` provides legitimate service, submits proofs (`tokensUsed` = 20,000)
3. **SlashingAuthority** discovers fraud in a **different** session
4. **SlashingAuthority** calls `slashStake(Host_A, amount, evidence, reason)`
5. `Host_A` loses 50% of stake but remains active (if `stake ≥ MIN_STAKE_AFTER_SLASH`)
6. `Host_A` continues submitting proofs in existing session
7. `Host_A` completes session and receives full payment

Result: Punished fraudulent host continues earning money, undermining the economic security model.

Attack Scenario 2: Auto-Unregistered Host After Severe Slash

1. Session exists between User and `Host_A`
2. `Host_A` is slashed, stake falls below `MIN_STAKE_AFTER_SLASH` (100 FAB)
3. `slashStake` triggers auto-unregister: delete `nodes[host]`
4. `Host_A` is completely removed from **NodeRegistry**
5. `Host_A` still can submit proofs and complete the session
6. `Host_A` receives payment despite being forcefully removed from the system

Assets:

- src/JobMarketplaceWithModelsUpgradeable.sol [https://github.com/Fabstir/fabstir-compute-contracts#]

Status: Accepted

Classification

Impact Rate: 4/5

Likelihood Rate: 3/5

Exploitability: Independent

Complexity: Simple

Severity: Medium

Recommendations

Remediation: Validate host status in `submitProofOfWork`:

```
require(nodeRegistry.isActiveNode(session.host), "Host no longer active");
```

Resolution: Accepted by development team:

No runtime host-activity enforcement is present in `submitProofOfWork` or `completeSessionJob`. Current logic still authorizes proof submission by `msg.sender == session.host` and permits settlement without checking `nodeRegistry.isActiveNode(session.host)`.

Host status validation remains limited to session creation paths, so previously created sessions can proceed after registry removal or deactivation.

F-2026-15144 - Dispute Window Calculated From Session Start Time Instead of Last Proof Submission - Medium

Description:

The `completeSessionJob` function uses `session.startTime` to calculate when the dispute window expires. This is either:

1. A bug: The implementation mistakenly uses startTime instead of lastProofTime
2. A design flaw: If intentional, comparing a global contract constant (`disputeWindow`) against per-session variable duration (`maxDuration`) is architecturally incorrect

```
if (msg.sender != session.depositor) {
    require(block.timestamp >= session.startTime + disputeWindow, "Must wait
dispute window");
}
```

The default `disputeWindow` is `> 0` seconds, as validated in initialize:

```
require(_disputeWindow > 0 && _disputeWindow <= 7 days, "Invalid dispute wind
ow");
```

The design compares incompatible values:

Variable	Type	Range
`disputeWindow`	Global constant	30s - 7 days (set once at deploy)
`maxDuration`	Per-session parameter	Variable (user chooses per session)

They are not comparable because:

- `disputeWindow` = 30 seconds is meaningless for a 24-hour session (expires at 0.03% of session)
- `disputeWindow` = 1 day blocks a 1-hour session from completing for 23 extra hours

The contract cannot know appropriate disputeWindow at deployment because session durations are determined later by users. A global constant cannot correctly protect sessions of varying lengths.

Attack Scenario

1. Session created with 100 USDC deposit at `T0`
2. Session runs normally for 1 hour with legitimate work
3. At `T0 + 1 hour`, host submits an inflated proof claiming excessive tokens

4. At `T0 + 1 hour + 1 second`, host calls `completeSessionJob`
5. Check passes: `T0 + 1 hour > T0 + 30 seconds`
6. Session completes immediately with inflated `tokensUsed`
7. User and the system had no opportunity to review the final proof

The dispute window becomes meaningless after the first `disputeWindow` seconds of a session. Proofs submitted after `startTime + disputeWindow` can be immediately settled with zero verification time.

Assets:

- src/JobMarketplaceWithModelsUpgradeable.sol [https://github.com/Fabstir/fabstir-compute-contracts#]

Status: Fixed

Classification

Impact Rate: 4/5

Likelihood Rate: 4/5

Exploitability: Semi-Dependent

Complexity: Simple

Severity: Medium

Recommendations

Remediation: Change the dispute window reference from `startTime` to `lastProofTime`:

```
if (msg.sender != session.depositor) {
        require(
            block.timestamp >= session.lastProofTime + disputeWindow,
            "Must wait dispute window after last proof"
        );
    }
```

If the current behavior is intentional, evaluate whether a single global constant can adequately protect sessions with varying durations, or if a per-session or proof-relative approach is needed.

Resolution: Fixed in `596ea59`:

`completeSessionJob` now gates non-depositor completion using `session.lastProofTime + disputeWindow` instead of `session.startTime + disputeWindow`.

```
if (msg.sender != session.depositor) {
    require(block.timestamp >= session.lastProofTime + disputeWindow, "Must w
ait dispute window");
}
```

The dispute delay is therefore anchored to the most recent proof submission, preserving a review interval before host-triggered settlement regardless of total session duration.

F-2026-15147 - Shared Proof Replay-Prevention State Across Multiple Uncoordinated Entry Points Causes Proof Submission Failure - Medium

Description:

The **ProofSystemUpgradeable** contract exposes multiple functions that mark a `proofHash` as consumed in the `verifiedProofs` mapping. If any of these functions processes a proof before `submitProofOfWork` does, the host's legitimate proof submission reverts, blocking payment for performed work.

Three functions in **ProofSystemUpgradeable** write `verifiedProofs[proofHash] = true`:

- `verifyAndMarkComplete` — no access control
- `verifyBatch` — no access control
- `recordVerifiedProof` — owner or authorized callers

The internal `_verifyHostSignature` function, used by all verification paths, rejects any proof whose hash is already in the mapping:

```
if (verifiedProofs[proofHash]) return false;
```

JobMarketplaceWithModelsUpgradeable.`submitProofOfWork` requires `verifyAndMarkComplete` to return `true`:

```
require(
    proofSystem.verifyAndMarkComplete(proof, msg.sender, tokensClaimed),
    "Invalid proof signature"
);
```

If the same `proofHash` has already been marked as verified through any of the three entry points, whether by an external caller, the owner, or an authorized contract, `verifyAndMarkComplete` returns `false` and `submitProofOfWork` reverts. The host cannot reuse or re-sign the same `proofHash`; it is permanently consumed. The only option is to generate an entirely new inference output off-chain, which is also subject to the same issue.

This is a design-level flaw: the proof lifecycle in **ProofSystemUpgradeable** does not account for the fact that `submitProofOfWork` is the only legitimate consumer of proof state transitions, yet multiple uncoordinated paths can trigger that same transition independently.

A host that has legitimately performed AI inference work is unable to submit proof and claim payment if the corresponding `proofHash` was

already consumed. The session eventually times out, settling with `tokensUsed = 0` for the blocked proofs, and the host receives no compensation for the computational resources expended. The renter's deposit is refunded. No deposited funds are lost, but the host suffers irrecoverable economic loss (wasted GPU resources) and the core payment-for-work guarantee of the marketplace is undermined.

Found in commit: `f614355`.

Assets:

- src/JobMarketplaceWithModelsUpgradeable.sol [https://github.com/Fabstir/fabstir-compute-contracts#]
- src/ProofSystemUpgradeable.sol [https://github.com/Fabstir/fabstir-compute-contracts#]

Status: Mitigated

Classification

Impact Rate: 3/5

Likelihood Rate: 4/5

Exploitability: Independent

Complexity: Simple

Severity: Medium

Recommendations

Remediation: It is recommended to consolidate all state-modifying proof operations into a single controlled path:

- `recordVerifiedProof` should be removed from **ProofSystemUpgradeable**. It provides an independent, uncoordinated way to mark proofs as consumed without going through the session lifecycle, which is fundamentally incompatible with the `submitProofOfWork` flow.
- `verifyAndMarkComplete` and `verifyBatch` should be restricted to authorized callers only (i.e., the **JobMarketplaceWithModelsUpgradeable** contract), ensuring that proof state transitions occur exclusively through `submitProofOfWork`.
- If the owner requires the ability to invalidate a proof outside the normal flow (e.g., for emergency or administrative purposes), a dedicated owner-only function should be introduced in **JobMarketplaceWithModelsUpgradeable** that calls **ProofSystemUpgradeable** to mark the proof as consumed **and**

performs the corresponding session-level state updates (incrementing `tokensUsed`, recording the `ProofSubmission` entry) to keep both contracts consistent.

Resolution: Mitigated in `0606d07`:

`ProofSystemUpgradeable` no longer exposes `verifyAndMarkComplete`, `verifyBatch`, or `recordVerifiedProof`. State mutation for replay protection is consolidated into `markProofUsed`, and access control is enforced through `authorizedCallers` or `owner`.

```
require(authorizedCallers[msg.sender] || msg.sender == owner(), "Unauthorized
");
```

This removes the previously unprotected external paths that enabled public front-running consumption of `proofHash`.
Full alignment with the recommended "marketplace-only consumer" model is not enforced at contract level, since `markProofUsed` remains callable by owner and any address added to `authorizedCallers`. An independent privileged path for proof consumption therefore remains.

Evidences

Proof of Concept: Proof consumed by multiple entry points

Reproduce:

```
function test_POC_ProofConsumedByMultipleEntryPoints() public {
        uint256 tokensClaimed = 1000;
        (bytes memory proof, bytes32 proofHash) = _createValidProof(tokensCla
imed, 100);


        assertFalse(proofSystemProxy.verifiedProofs(proofHash));


        vm.prank(attacker);
        bool attackerResult = proofSystemProxy.verifyAndMarkComplete(proof, h
ost, tokensClaimed);
        assertTrue(attackerResult);
        assertTrue(proofSystemProxy.verifiedProofs(proofHash));


        vm.prank(authorizedCaller);
        bool marketplaceResult = proofSystemProxy.verifyAndMarkComplete(proof
, host, tokensClaimed);
        assertFalse(marketplaceResult);
```

```
        vm.prank(owner);
        bool ownerResult = proofSystemProxy.verifyAndMarkComplete(proof, host
, tokensClaimed);
        assertFalse(ownerResult);


        bytes[] memory batchProofs = new bytes[](1);
        uint256[] memory batchTokens = new uint256[](1);
        batchProofs[0] = proof;
        batchTokens[0] = tokensClaimed;


        vm.prank(attacker);
        vm.expectRevert("Invalid proof at index");
        proofSystemProxy.verifyBatch(batchProofs, host, batchTokens);
    }
```

Results:

```
[PASS] test_POC_ProofConsumedByMultipleEntryPoints()
```

F-2026-15149 - Missing Access Control on verifyAndMarkComplete Enables Proof Griefing Attack - Medium

Description:

The **ProofSystemUpgradeable** contract implements access control for `recordVerifiedProof` but fails to implement the same protection for `verifyAndMarkComplete` and `verifyBatch`.

The `recordVerifiedProof` function requires authorization:

```
function recordVerifiedProof(bytes32 proofHash) external {
        require(authorizedCallers[msg.sender] || msg.sender == owner(), "Unau
thorized");
        verifiedProofs[proofHash] = true;
        emit ProofVerified(proofHash, msg.sender, 0);
    }
```

However, `verifyAndMarkComplete` has no access control:

```
 function verifyAndMarkComplete(bytes calldata proof, address prover, uint256
claimedTokens)
        external
        returns (bool)
    {
        // First verify using internal function
        if (!_verifyHostSignature(proof, prover, claimedTokens)) {
            return false;
        }

        ...

        verifiedProofs[proofHash] = true;
        emit ProofVerified(proofHash, prover, claimedTokens);

        return true;
    }
```

Similarly, `verifyBatch` lacks access control:

```
function verifyBatch(bytes[] calldata proofs, address prover, uint256[] calld
ata tokenCounts)
        external
        returns (bool)
    {
        ...
```

```
        for (uint256 i = 0; i < proofs.length; i++) {
            // Verify each proof using internal function
            require(_verifyHostSignatureInternal(proofs[i], prover, tokenCoun
ts[i]), "Invalid proof at index");

            ...
            proofHashes[i] = proofHash;
            verifiedProofs[proofHash] = true;
            totalTokens += tokenCounts[i];
        }


        emit BatchProofVerified(proofHashes, prover, totalTokens);
        return true;
    }
```

The replay prevention mechanism (`verifiedProofs` mapping) becomes a denial-of-service vector when anyone can mark proofs as “verified.”

Front-Running Griefing Attack Scenario:

1. Host creates a valid proof and signs it
2. Host submits `submitProofOfWork` transaction to mempool
3. Attacker observes the pending transaction and extracts proof data
4. Attacker front-runs with direct call to proofSystem.verifyAndMarkComplete(proof, host, tokens)
5. Attacker's transaction executes first, marking verifiedProofs[proofHash] = true
6. Host's submitProofOfWork() transaction executes but fails at the replay check in _verifyHostSignature(): `if (verifiedProofs[proofHash]) return false;`
7. Host's legitimate proof submission reverts with "Invalid proof signature"
8. Host must generate a completely new proof with different data to try again
9. Attacker can repeat this indefinitely, preventing the host from ever submitting proofs

Attackers can prevent hosts from submitting valid proofs by front-running and "burning" their proof hashes. Hosts unable to submit proofs cannot earn payment for legitimate work performed.

Assets:

- src/JobMarketplaceWithModelsUpgradeable.sol [https://github.com/Fabstir/fabstir-compute-contracts#]
- src/ProofSystemUpgradeable.sol [https://github.com/Fabstir/fabstir-compute-contracts#]

Status: Fixed

Classification

Impact Rate: 3/5

Likelihood Rate: 4/5

Exploitability: Independent

Complexity: Simple

Severity: Medium

Recommendations

Remediation: Add access control to `verifyAndMarkComplete` and `verifyBatch` to ensure only **JobMarketplaceWithModelsUpgradeable** contract can call them.

Resolution: Fixed in `0606d07`:

`verifyAndMarkComplete` and `verifyBatch` are no longer present in **ProofSystemUpgradeable**. The replay-protection write path has been consolidated into `markProofUsed`, which enforces access control through `authorizedCallers[msg.sender] || msg.sender == owner()`.

```
function markProofUsed(
    bytes32 proofHash,
    address prover,
    uint256 claimedTokens,
    bytes32 modelId
) external override returns (bool) {
    require(authorizedCallers[msg.sender] || msg.sender == owner(), "Unauthor
ized");

    ...
}
```

JobMarketplaceWithModelsUpgradeable now calls `markProofUsed` during proof submission, removing the previously exposed public verification endpoints that enabled front-running griefing through unauthorized proof pre-marking.

Evidences

Proof of Concept: Front-running Griefing Attack

Reproduce:

```
function test_POC_FrontRunningGriefingAttack() public {
        uint256 tokensClaimed = 1000;
        (bytes memory proof, ) = _createValidProof(tokensClaimed, 2);

        vm.prank(attacker);
        bool attackResult = proofSystemProxy.verifyAndMarkComplete(proof, hos
t, tokensClaimed);
        assertTrue(attackResult);

        vm.prank(authorizedCaller);
        bool hostResult = proofSystemProxy.verifyAndMarkComplete(proof, host,
tokensClaimed);

        assertFalse(hostResult);
    }
```

Results:

[PASS] test_POC_FrontRunningGriefingAttack()

Proof of Concept: verifyBatch lacks access control

Reproduce:

```
function test_POC_VerifyBatchLacksAccessControl() public {
        uint256 tokensClaimed = 500;

        bytes[] memory proofs = new bytes[](2);
        uint256[] memory tokenCounts = new uint256[](2);

        (bytes memory proof1, bytes32 hash1) = _createValidProof(tokensClaime
d, 10);
        proofs[0] = proof1;
        tokenCounts[0] = tokensClaimed;

        (bytes memory proof2, bytes32 hash2) = _createValidProof(tokensClaime
d, 20);
        proofs[1] = proof2;
        tokenCounts[1] = tokensClaimed;

        vm.prank(attacker);
        bool result = proofSystemProxy.verifyBatch(proofs, host, tokenCounts)
;

        assertTrue(result);
        assertTrue(proofSystemProxy.verifiedProofs(hash1));
```

```
        assertTrue(proofSystemProxy.verifiedProofs(hash2));
    }
```

Results:

```
[PASS] test_POC_VerifyBatchLacksAccessControl()
```

F-2026-15162 - Deactivated Model Still Usable for Sessions and Node Registration - Medium

Description:

When the owner deactivates a model via `deactivateModel`, no enforcement exists in the session creation path to prevent new sessions from being created against that model. The deactivation mechanism is effectively bypassed.

`createSessionJobForModel` and `createSessionJobForModelWithToken` validate that a host supports a model by calling `nodeRegistry.nodeSupportsModel()`, which only checks the host's local `supportedModels` array:

```
    function nodeSupportsModel(address nodeAddress, bytes32 modelId) external
view returns (bool) {
        bytes32[] memory models = nodes[nodeAddress].supportedModels;
        for (uint i = 0; i < models.length; i++) {
            if (models[i] == modelId) return true;
        }
        return false;
    }
```

Neither the **JobMarketplaceWithModelsUpgradeable** session creation functions nor `nodeSupportsModel` ever call `modelRegistry.isModelApproved(modelId)` to verify the model is still active:

JobMarketplaceWithModelsUpgradeable

```
        require(nodeRegistry.nodeSupportsModel(host, modelId), "Host does not
support model");
```

Model approval is checked only at node registration time (`registerNode`) and model update time (`updateSupportedModels`), but never at session creation. Once a host has the model in its local array, deactivation in **ModelRegistry** has no downstream effect.

A model deactivated by the owner for security reasons (e.g., compromised weights, harmful outputs) continues to be available for new session creation indefinitely. This renders the emergency deactivation mechanism ineffective and exposes renters to inference from a model the owner explicitly deemed unsafe.

Assets:

- src/JobMarketplaceWithModelsUpgradeable.sol [https://github.com/Fabstir/fabstir-compute-contracts#]

- src/NodeRegistryWithModelsUpgradeable.sol [https://github.com/Fabstir/fabstir-compute-contracts#]
- src/ModelRegistryUpgradeable.sol [https://github.com/Fabstir/fabstir-compute-contracts#]

Status: Fixed

Classification

Impact Rate: 4/5

Likelihood Rate: 3/5

Exploitability: Independent

Complexity: Simple

Severity: Medium

Recommendations

Remediation: Add an `isModelApproved` check in **JobMarketplaceWithModelsUpgradeable** during model-aware session creation, querying the **ModelRegistry** directly. Expose a reference to **ModelRegistry** (either stored on JobMarketplace or accessed via `nodeRegistry.modelRegistry()`) and insert the check before the host-support validation:

```
require(nodeRegistry.modelRegistry().isModelApproved(modelId), "Model not active");
require(nodeRegistry.nodeSupportsModel(host, modelId), "Host does not support model");
```

Apply this to both `createSessionJobForModel` and `createSessionJobForModelWithToken`.

Resolution: Fixed in `a49ef12`:

JobMarketplaceWithModelsUpgradeable now enforces model activation at model-aware session creation by requiring `nodeRegistry.modelRegistry().isModelApproved(modelId)` before `nodeRegistry.nodeSupportsModel(host, modelId)`.

```
require(nodeRegistry.modelRegistry().isModelApproved(modelId), "Model not approved");
```

The control is present in `createSessionJobForModel` and `createSessionJobForModelWithToken`, and the same enforcement is applied in the delegate flow `createSessionForModelAsDelegate`.

F-2026-15257 - Early Cancel Fee Applied on Depositor-Triggered Timeouts - Medium

Description:

The `_settleSessionPayments` function applies the early cancellation fee based solely on whether the caller (`completedBy`) matches the session depositor and no proofs exist. This logic is shared by both `completeSessionJob` (voluntary termination) and `triggerSessionTimeout` (forced termination due to host inactivity). No distinction is made between the two termination paths.

```
uint256 earlyFee;
// Early cancel fee: depositor cancels before any proofs
if (completedBy == session.depositor && session.proofs.length == 0 && minTokensFee > 0) {
            earlyFee = (minTokensFee * session.pricePerToken) / PRICE_PRECISION;
            if (hostPayment >= session.deposit) {
                earlyFee = 0;
            } else if (earlyFee > session.deposit - hostPayment) {
                earlyFee = session.deposit - hostPayment;
            }
        }
```

`triggerSessionTimeout` passes `msg.sender` as `completedBy`:

```
_settleSessionPayments(jobId, msg.sender);
```

When a host fails to submit any proof within `proofTimeoutWindow`, the session becomes eligible for timeout. If the depositor calls `triggerSessionTimeout`, the condition `completedBy == session.depositor && session.proofs.length == 0 && minTokensFee > 0` evaluates to `true`, and the early cancellation fee is deducted from the depositor's deposit and forwarded to the host.

This inverts the intended economic incentive: the early cancel fee exists to penalize depositors who create and immediately cancel sessions, wasting host resources. In the timeout scenario, the host abandoned the session by not submitting proofs. Charging the depositor for the host's inaction and forwarding that fee to the inactive host rewards non-performance.

Found in commit: `f614355`.

Assets:

- src/JobMarketplaceWithModelsUpgradeable.sol [https://github.com/Fabstir/fabstir-compute-contracts#]

Status: Fixed

Classification

Impact Rate: 3/5

Likelihood Rate: 4/5

Exploitability: Independent

Complexity: Simple

Severity: Medium

Recommendations

Remediation: Restrict early cancel fee application to voluntary session completion only. One approach is to check the session status at the point of fee evaluation:

```
if (completedBy == session.depositor && session.proofs.length == 0
  && minTokensFee > 0 && session.status == SessionStatus.Completed) {
```

Since `triggerSessionTimeout` sets `session.status = SessionStatus.TimedOut` before calling `_settleSessionPayments`, the condition would exclude timeout-triggered settlements while preserving fee application for voluntary depositor cancellations via `completeSessionJob`.

Resolution: The early cancellation fee condition now additionally checks that `session.status == SessionStatus.Completed`, ensuring the fee is only applied on voluntary depositor cancellations and not on timeout paths where the session status is `TimedOut`:

```
// Early cancel fee: depositor cancels before any proofs (F202615257: not on
timeout)
if (completedBy == session.depositor && session.status == SessionStatus.Compl
eted && session.proofs.length == 0 && minTokensFee > 0) {
```

Revised commit: df1f2e4.

F-2026-15280 - Hardcoded Private Key Exposure in Test Scripts Enables Unauthorized Account Control - Medium

Description:

Private keys were embedded in repository test scripts, which constitutes credential exposure and enables full signing authority over the corresponding externally owned accounts. Since both derived addresses were observed with on-chain activity in October 2025, the exposure cannot be treated as purely synthetic test data and must be considered an operational security incident until key rotation and usage invalidation are confirmed.

The integration/example JavaScript tests instantiate wallets from literal private-key constants, creating recoverable signing credentials in source control history.

Representative pattern (redacted):

```
// Wallets
const userWallet = new ethers.Wallet('<REDACTED_PRIVATE_KEY>', provider);
const hostWallet = new ethers.Wallet('<REDACTED_PRIVATE_KEY>', provider);
```

Affected locations include:

- tests/integration/usdc-payment-flow.js
- tests/integration/usdc-complete-cycle.js
- tests/examples/usdc-job-parsing-example.js

Derived addresses:

- `0x8D64…..4bF6`
- `0x4594…..4504`

Risk mechanism:

- Any actor with repository access can derive the same accounts and produce valid signatures.
- If the same credentials were reused in deployment pipelines, operational wallets, relayers, or privileged off-chain services, unauthorized transactions and state changes could be executed.
- Exposure persists in Git history even after file-level replacement, unless history rewrite and coordinated secret revocation are performed.

If key reuse occurred in any operational context, unauthorized actors could execute transactions, impersonate trusted signers, and transfer controlled assets; even where current deployment usage is dismissed and keys are rotated, historical leakage preserves residual risk until complete revocation validation and dependency rotation are finalized.

Found in commit: `f614355`.

Status: Fixed

Classification

Impact Rate: 5/5

Likelihood Rate: 3/5

Exploitability: Dependent

Complexity: Simple

Severity: Medium

Recommendations

Remediation:

- Immediate revocation/retirement of all exposed keypairs should be enforced.
- Verification should be completed that no deployment, relayer, automation job, CI secret, or production environment references the exposed addresses.
- Fresh keys should be generated through secure entropy sources and isolated per environment (dev/test/staging/prod) with strict non-reuse policy.
- Secret material should be loaded from secure secret managers or environment injection, never from literals in source code.
- Repository and CI secret-scanning controls should be enabled (pre-commit and pipeline) to block future commits containing private-key patterns.

Resolution: Hardcoded private keys have been replaced with environment variable references across all affected test scripts; however, the original keys persist in Git history and the associated accounts should be considered permanently compromised — key rotation and fund migration are still required. Revised commit: `df1f2e4`.

F-2026-14909 - Proof Verification Bypassed When ProofSystem Is Not Configured - Low

Description:

The `submitProofOfWork()` function conditionally verifies proof submissions based on whether the `proofSystem` address is configured. When `proofSystem` is set to `address(0)`, the entire cryptographic verification logic is skipped, allowing hosts to submit arbitrary `proofHash` and `signature` values without any validation. Although hosts must be registered with staked tokens and can be slashed for misbehavior, the absence of on-chain proof verification removes the primary mechanism for detecting fraudulent token claims, leaving only post-hoc manual review and slashing as deterrents.

```
function submitProofOfWork(
    uint256 jobId,
    uint256 tokensClaimed,
    bytes32 proofHash,
    bytes calldata signature,
    string calldata proofCID,
    string calldata deltaCID
) external nonReentrant whenNotPaused {
    SessionJob storage session = sessionJobs[jobId];
    require(session.status == SessionStatus.Active, "Session not active");
    require(msg.sender == session.host, "Only host can submit proof");
    require(tokensClaimed >= MIN_PROVEN_TOKENS, "Must claim minimum tokens");
    require(signature.length == 65, "Invalid signature length");
    // ... rate limiting and deposit checks ...
    // Verification is completely skipped when proofSystem == address(0)
    bool verified = false;
    if (address(proofSystem) != address(0)) {
        bytes memory proof = abi.encodePacked(proofHash, signature);
        require(
            proofSystem.verifyAndMarkComplete(proof, msg.sender, tokensClaimed
),
            "Invalid proof signature"
        );
        verified = true;
    }
    // Proof is recorded and tokens are credited regardless of verification
    session.proofs.push(
        ProofSubmission({
            proofHash: proofHash,
            tokensClaimed: tokensClaimed,
            timestamp: block.timestamp,
            verified: verified,  // Set to false when proofSystem not configur
ed
```

```
            deltaCID: deltaCID
        })
    );
    session.tokensUsed = newTotal;  // Tokens credited without verification
    session.lastProofTime = block.timestamp;
    emit ProofSubmitted(jobId, msg.sender, tokensClaimed, proofHash, proofCID,
deltaCID);
}
```

Impact:

- Registered hosts can claim tokens without performing actual AI inference work
- The verified flag is stored as false but settlement proceeds normally based on `tokensUsed`
- Users have no on-chain protection against fraudulent claims when `proofSystem` is not configured

Assets:

- src/JobMarketplaceWithModelsUpgradeable.sol [https://github.com/Fabstir/fabstir-compute-contracts#]

Status: Fixed

Classification

Impact Rate: 4/5

Likelihood Rate: 2/5

Exploitability: Semi-Dependent

Complexity: Simple

Severity: Low

Recommendations

Remediation: Require `proofSystem` to be configured before allowing proof submissions.

Resolution: Fixed in `c175fa6`:

`submitProofOfWork` now enforces **ProofSystem** configuration at entry by requiring `address(proofSystem) != address(0)`.
Proof submission no longer proceeds when verifier integration is unset, removing the prior conditional bypass where claims could be recorded without cryptographic verification.

```
require(address(proofSystem) != address(0), "ProofSystem not set");
```

Token accounting updates in proof submission are therefore gated behind configured replay/verification flow.

F-2026-14913 - Hardcoded Token Rate Limit Cannot Adapt to Future Hardware Improvements - Low

Description:

The `submitProofOfWork()` function enforces a hardcoded maximum token claim rate of 2000 tokens per second as an anti-fraud sanity check. While this limit is intentionally set well above current LLM inference speeds (20-500 tokens/sec), it cannot be adjusted without a contract upgrade. As AI hardware continues to advance rapidly, this fixed rate may become a bottleneck that prevents legitimate high-performance hosts from claiming tokens at their actual throughput rate. Additionally, this fixed rate ignores that different models may have different speeds.

```
    function submitProofOfWork(...) external nonReentrant whenNotPaused {
      // ... rest of the code
      // Rate limit: 1000 tokens/sec base * 2x buffer = 2000 tokens/sec max
      uint256 expectedTokens = timeSinceLastProof * 1000;
      require(tokensClaimed <= expectedTokens * 2, "Excessive tokens claimed
");
      uint256 newTotal = session.tokensUsed + tokensClaimed;
      // With PRICE_PRECISION: maxTokens = deposit * PRICE_PRECISION / price
PerToken
      uint256 maxTokens = (session.deposit * PRICE_PRECISION) / session.pric
ePerToken;
      require(newTotal <= maxTokens, "Exceeds deposit");
     // ... rest of the code
   }
```

- As hardware improves, the 2000 tokens/sec limit may become restrictive
- Different models have vastly different inference speeds - a one-size-fits-all limit is not optimal
- Adjusting the rate requires a full contract upgrade through the UUPS proxy pattern, which is operationally expensive and introduces upgrade risks

Assets:

- src/JobMarketplaceWithModelsUpgradeable.sol [https://github.com/Fabstir/fabstir-compute-contracts#]

Status: Fixed

Classification

Impact Rate: 3/5

Likelihood Rate: 2/5

Exploitability: Independent

Complexity: Simple

Severity: Low

Recommendations

Remediation: Make the rate limit configurable per model to reflect actual expected performance:

```
mapping(bytes32 => uint256) public modelMaxTokensPerSecond;
uint256 public defaultMaxTokensPerSecond = 2000;
function setModelMaxTokensPerSecond(bytes32 modelId, uint256 rate) external o
nlyOwner {
   require(rate >= 0, "Rate too low");
   modelMaxTokensPerSecond[modelId] = rate;
   emit ModelMaxTokensPerSecondUpdated(modelId, rate);
}
```

Resolution: Fixed in `d41c9f6`:

The hardcoded throughput check was replaced by model-aware rate limiting. `submitProofOfWork` now obtains the active model rate from `nodeRegistry.modelRegistry().getModelRateLimit(modelId)` and enforces claims against that value instead of a fixed global constant.

```
bytes32 modelId = sessionModel[jobId];
uint256 maxRate = nodeRegistry.modelRegistry().getModelRateLimit(modelId);
uint256 expectedTokens = timeSinceLastProof * maxRate;
require(tokensClaimed <= expectedTokens, "Excessive tokens claimed");
```

ModelRegistryUpgradeable now provides owner-managed per-model rate configuration, enabling operational tuning without marketplace contract upgrade for each throughput change.

```
function setModelRateLimit(bytes32 modelId, uint256 maxTokensPerSec) external
onlyOwner {
    require(models[modelId].timestamp > 0, "Model does not exist");
    require(maxTokensPerSec >= MIN_RATE_LIMIT, "Rate below minimum");
    require(maxTokensPerSec <= MAX_RATE_LIMIT, "Rate above maximum");
    modelMaxTokensPerSecond[modelId] = maxTokensPerSec;
    emit ModelRateLimitUpdated(modelId, maxTokensPerSec);
```

```
}

function getModelRateLimit(bytes32 modelId) external view returns (uint256) {
    uint256 rate = modelMaxTokensPerSecond[modelId];
    return rate > 0 ? rate : DEFAULT_RATE_LIMIT;
}
```

F-2026-14917 - Depositors Can Cancel Sessions Immediately - No Protection Window for Hosts - Low

Description:

The **JobMarketplaceWithModelsUpgradeable** contract allows depositors to complete active sessions immediately after creation via `completeSessionJob` with no time restriction. Additionally, the `maxDuration` parameter has no minimum floor (only requires > 0), allowing depositors to set extremely short durations that make meaningful work impossible.

This creates an asymmetric situation where depositors have full control over session timing, while hosts have no guaranteed window to perform work and claim payment. Combined with mempool visibility, this enables front-running attacks where depositors can observe host proof submissions and complete the session before proofs are recorded.

Depositor can complete immediately - no grace period for hosts:

```
 function completeSessionJob(uint256 jobId, string calldata conversationCID) e
xternal nonReentrant {
        SessionJob storage session = sessionJobs[jobId];
        require(session.status == SessionStatus.Active, "Session not active")
;

        // Only depositor or host can complete and set conversationCID
        require(
            msg.sender == session.depositor || msg.sender == session.host,
            "Only depositor or host can complete"
        );

        // Dispute window only waived for the original depositor
        if (msg.sender != session.depositor) {
            require(block.timestamp >= session.startTime + disputeWindow, "Mu
st wait dispute window");
        }

        session.status = SessionStatus.Completed;
        session.conversationCID = conversationCID;

        _settleSessionPayments(jobId, msg.sender);
    }
```

The `disputeWindow` check (line 673-675) only applies when `msg.sender != session.depositor.` The depositor bypasses this check entirely and can complete the session in the same block it was created.

Impact:

- **Resource Waste**: Hosts expend compute resources and electricity for sessions that depositors can cancel without payment
- **Economic Griefing**: Malicious users can repeatedly create and cancel sessions to drain host resources
- **Front-Running Vulnerability**: Depositors can monitor mempool and front-run host proof submissions
- **Asymmetric Risk**: Hosts bear all the risk (resources spent) while depositors have full control over timing

Assets:

- src/JobMarketplaceWithModelsUpgradeable.sol [https://github.com/Fabstir/fabstir-compute-contracts#]

Status: Mitigated

Classification

Impact Rate: 3/5

Likelihood Rate: 2/5

Exploitability: Independent

Complexity: Simple

Severity: Low

Recommendations

Remediation:

- Add a minimum grace period before cancellation is allowed

```
uint256 public constant MIN_SESSION_DURATION = 5 minutes;
```

- Enforce grace period for depositor cancellation:

```
function completeSessionJob(uint256 jobId, string calldata conversationCID) e
xternal nonReentrant {
    SessionJob storage session = sessionJobs[jobId];
    require(session.status == SessionStatus.Active, "Session not active");
    require(
        msg.sender == session.depositor || msg.sender == session.host,
        "Only depositor or host can complete"
    );
    // Both depositor and host must wait minimum session duration
    require(
        block.timestamp >= session.startTime + MIN_SESSION_DURATION,
```

```
        "Cannot complete during grace period"
    );
    // Additional dispute window for non-depositor
    if (msg.sender != session.depositor) {
        require(block.timestamp >= session.startTime + disputeWindow, "Must wa
it dispute window");
    }
    // ...
}
```

- Enforce minimum `maxDuration` floor:

```
function _validateSessionParams(...) internal pure {
    // ...
    require(maxDuration >= MIN_SESSION_DURATION, "Duration too short");
}
```

Resolution: Mitigated in `81c5021`:

Immediate depositor-driven completion remains permitted, and `maxDuration` still has no minimum floor beyond `> 0`, so the timing asymmetry described in the finding is not fully removed.

Risk is reduced by the introduced early-cancellation charge path (`minTokensFee`) applied when a depositor completes before any proof submission, which creates a configurable economic disincentive against zero-work cancellations.

```
uint256 earlyFee;
// Early cancel fee: depositor cancels before any proofs
if (completedBy == session.depositor && session.proofs.length == 0 && minToke
nsFee > 0) {
    earlyFee = (minTokensFee * session.pricePerToken) / PRICE_PRECISION;
    if (hostPayment >= session.deposit) {
        earlyFee = 0;
    } else if (earlyFee > session.deposit - hostPayment) {
        earlyFee = session.deposit - hostPayment;
    }
}
```

F-2026-14965 - lateVotes Not Cleared on Proposal Execution - Low

Description:

The `lateVotes` mapping tracks cumulative votes cast during the final 4-hour "extension window" for anti-sniping protection. When 10,000+ FAB tokens are voted in this window, the voting period is extended by 1 day.

However, `lateVotes[modelId]` is never cleared when a proposal is executed or rejected. When a rejected model is re-proposed after the 30-day cooldown, the new proposal inherits the stale `lateVotes` value from the previous proposal.

This causes the anti-sniping extension to trigger with fewer votes than intended, or potentially trigger immediately upon the first late vote.

`lateVotes` accumulated but never reset on execution:

```
function voteOnProposal(bytes32 modelId, uint256 amount, bool support) extern
al nonReentrant {
    // ...
    uint256 timeUntilEnd = proposal.endTime - block.timestamp;
    if (timeUntilEnd <= EXTENSION_WINDOW) {
        lateVotes[modelId] += amount;

        if (lateVotes[modelId] >= EXTENSION_THRESHOLD && ...) {
            proposal.endTime += EXTENSION_DURATION;
            lateVotes[modelId] = 0;  // Only reset on extension trigger
        }
    }
}
```

`_clearOldProposal()` does not clear `lateVotes`:

```
function _clearOldProposal(bytes32 modelId) internal {
    if (proposals[modelId].executed) {
        delete proposals[modelId];
    }
    require(proposals[modelId].endTime == 0, "Active proposal exists");
}
```

Anti-sniping mechanism behaves incorrectly for re-proposed models
Extensions can trigger with fewer votes than intended

Assets:

- src/ModelRegistryUpgradeable.sol [https://github.com/Fabstir/fabstir-compute-contracts#]

Status: Fixed

Classification

Impact Rate: 2/5

Likelihood Rate: 2/5

Exploitability: Independent

Complexity: Simple

Severity: Low

Recommendations

Remediation: Clear `lateVotes` when clearing old proposal data:

```
function _clearOldProposal(bytes32 modelId) internal {
   if (proposals[modelId].executed) {
       delete proposals[modelId];
       delete lateVotes[modelId];
   }
   require(proposals[modelId].endTime == 0, "Active proposal exists");
}
```

Resolution: Fixed in `2d7256c`:

ModelRegistryUpgradeable now clears stale anti-sniping state during re-proposal cleanup by deleting `lateVotes[modelId]` inside `_clearOldProposal` when prior proposal data is removed.

```
if (proposals[modelId].executed) {
    delete proposals[modelId];
    delete lateVotes[modelId];  // Clear stale late votes for fresh re-propos
al
}
```

F-2026-14998 - Signature Malleability Attack in ProofSystemUpgradeable Contract - Low

Description:

The `_verifyHostSignature` function uses `ecrecover` to validate the host's ECDSA signature over the proof data but does not enforce a canonical (low-s) form of the signature. ECDSA signatures are malleable: both (r, s) and (r, n - s) are valid for the same message and recover to the same address. The contract accepts either form.

In this design, replay is keyed only by `proofHash`, so both encodings share the same hash and only one can ever be accepted; there is no double use or replay.

```
function _verifyHostSignature(bytes calldata proof, address prover, uint256 c
laimedTokens) internal view returns (bool) {
    ...

    bytes32 proofHash;
    bytes32 r;
    bytes32 s;
    uint8 v;

    assembly {
        proofHash := calldataload(proof.offset)
        r := calldataload(add(proof.offset, 32))
        s := calldataload(add(proof.offset, 64))
        v := byte(0, calldataload(add(proof.offset, 96)))
    }

    // Check not already verified (prevent replay)
    if (verifiedProofs[proofHash]) return false;

    bytes32 dataHash = keccak256(abi.encodePacked(proofHash, prover, claimedT
okens));
    bytes32 messageHash = keccak256(abi.encodePacked("\x19Ethereum Signed Mes
sage:\n32", dataHash));

    // Recover signer and verify it matches the prover (host)
    address recoveredSigner = ecrecover(messageHash, v, r, s);

    ...
}
```

The inconsistency does not point a direct vulnerability, it outlines best-practice violation. Risk is limited to consistency with standards and

possible future reuse of the logic in a context where the full signature is used as a replay key; in that case, accepting malleable signatures could become relevant.

Assets:

- src/ProofSystemUpgradeable.sol [https://github.com/Fabstir/fabstir-compute-contracts#]

Status: Fixed

Classification

Impact Rate: 3/5

Likelihood Rate: 2/5

Exploitability: Independent

Complexity: Simple

Severity: Low

Recommendations

Remediation: To enhance the security of your Solidity smart contracts and mitigate the risk of signature malleability attacks, it is advisable to use OpenZeppelin's ECDSA library instead of the built-in `ecrecover()` function. The ECDSA library provides robust and reliable signature verification, reducing the vulnerability to replay attacks and ensuring the integrity of the contract interactions.

Resolution: Fixed in `0606d07`:

Signature-recovery logic based on ecrecover is no longer present in **ProofSystemUpgradeable**. Legacy paths that parsed and verified (`r, s, v`) signatures were removed with the deprecated verification entry points, and replay protection now relies on controlled proof-hash consumption via `markProofUsed`.

F-2026-15064 - Unfinalized Access-Control Functionality in ProofSystemUpgradeable Contract - Low

Description:

The `ProofSystemUpgradeable` contract introduces the `authorizedCallers` mapping which entries are manually set by the owner. The only function allowed for the authorized accounts to invoke is `recordVerifiedProof` function allowing to mark arbitrary proof as used.

According to the NatSpec, the function aims to participate in Replay Attacks prevention and the `authorizedCallers` mapping is supposed to enable system contracts for the function execution.

```
/**
 * @notice Record a verified proof to prevent replay attacks
 * @dev Only callable by authorized contracts (e.g., JobMarketplace) or owner
 * @param proofHash The hash of the verified proof
 */
```

However, the system contracts never execute the `recordVerifiedProof` function. The function does not participate in Replay Attacks prevention as manual disabling of particular proofs is not a part of an automatic workflow.

The inconsistency does not point a direct vulnerability, it outlines a redundant functionality trying to mislead auditors and developers about how the `ProofSystemUpgradeable` operates, particularly, the functionality introduces access-control features which are not properly integrated in the contract.

Assets:

- src/ProofSystemUpgradeable.sol [https://github.com/Fabstir/fabstir-compute-contracts#]

Status: Fixed

Classification

Impact Rate: 2/5

Likelihood Rate: 3/5

Exploitability: Independent

Complexity: Simple

Severity: Low

Recommendations

Remediation: Consider removing the redundant `recordVerifiedProof` endpoint and properly integrating access-control to the needed functions according to architecture.

Resolution: Fixed in `0606d07`:

The redundant proof-recording surface was removed as part of the **ProofSystem** simplification. `recordVerifiedProof` is no longer present, and replay-state mutation is centralized into `markProofUsed` with active access control through `authorizedCallers` / `owner`.

F-2026-15255 - Delegate Authorization Has No Spending Limits - Low

Description:

Delegate authorization is binary (`authorized` `true`/`false`) and does not include budget controls.

```
function isDelegateAuthorized(address depositor, address delegate) external view returns (bool) {
    return isAuthorizedDelegate[depositor][delegate];
}
```

Once authorized, a delegate can create sessions on behalf of the payer using any amount permitted by protocol-level bounds and the payer's token allowance to the marketplace. There is no payer-level control for:

- maximum spend per session,
- cumulative spend limit per delegate,
- expiration for delegate permissions.

An authorized delegate can consume a payer's approved funds at scale, constrained only by token allowance and global min/max deposit rules. This creates excessive trust requirements and increases exposure in delegate key compromise or malicious delegate scenarios.

Assets:

- src/JobMarketplaceWithModelsUpgradeable.sol [https://github.com/Fabstir/fabstir-compute-contracts#]

Status: Fixed

Classification

Impact Rate: 3/5

Likelihood Rate: 2/5

Exploitability: Semi-Dependent

Complexity: Simple

Severity: Low

Recommendations

Remediation: Introduce payer-configurable delegate limits, for example:

- `maxPerSession`,

- `totalCap`,
- `spent`,
- optional `validUntil` timestamp.

Enforce these limits in createSessionForModelAsDelegate before token transfer.

Resolution: `DelegateConfig` now includes all the recommended fields (`maxPerSession`, `totalCap`, `spent`, `validUntil`) and they are enforced in `createSessionForModelAsDelegate()` before the token transfer. Revised commit: `df1f2e4`.

F-2026-15256 - Delegate Scope Is Not Restrictable by Host or Model - Low

Description: Delegate permissions are global for a payer and cannot be scoped to specific hosts or models. An authorized delegate can create sessions with any registered host and any approved model.

```
function authorizeDelegate(address delegate, bool authorized) external {
    require(delegate != address(0), "Invalid delegate address");
    require(delegate != msg.sender, "Cannot delegate to self");

    isAuthorizedDelegate[msg.sender][delegate] = authorized;
    emit DelegateAuthorized(msg.sender, delegate, authorized);
}
```

Permission granularity is too coarse for operational delegation use cases (e.g., service-specific sub-accounts). A delegate intended for one provider/model can route sessions through any available host/model combination.

Assets:

- src/JobMarketplaceWithModelsUpgradeable.sol [https://github.com/Fabstir/fabstir-compute-contracts#]

Status: Mitigated

Classification

Impact Rate: 3/5

Likelihood Rate: 2/5

Exploitability: Semi-Dependent

Complexity: Simple

Severity: Low

Recommendations

Remediation: Add optional scope controls per delegate:

- allowlisted hosts (`payer => delegate => host => bool`),
- allowlisted models (`payer => delegate => modelId => bool`),
- default-deny behavior when scoped mode is enabled.

Enforce scope checks in createSessionForModelAsDelegate.

Resolution: The original finding is resolved — delegates can now be scoped to a specific host and model via `DelegateConfig`. However, only a single `allowedHost` and `allowedModel` can be set per delegate; payers who need to permit multiple (but not all) hosts or models must either grant unrestricted access or create separate delegate entries.

```
    function configureDelegate(
        address delegate,
        uint128 maxPerSession,
        uint128 totalCap,
        uint64 validUntil,
        address allowedHost,
        bytes32 allowedModel
    ) external {
        require(delegate != address(0), "Zero addr");
        require(delegate != msg.sender, "Self deleg");
        delegateConfigs[msg.sender][delegate] = DelegateConfig({
            maxPerSession: maxPerSession,
            totalCap: totalCap,
            spent: 0,
            validUntil: validUntil,
            active: true,
            allowedHost: allowedHost,
            allowedModel: allowedModel
        });
        emit DelegateAuthorized(msg.sender, delegate, true);
        emit DelegateConfigured(msg.sender, delegate, maxPerSession, totalCap,
validUntil, allowedHost, allowedModel);
    }
```

Revised commit: `df1f2e4`.

F-2026-15258 - setMinTokensFee Function Has No Upper Bound - Low

Description:

The `setMinTokensFee` admin function allows setting `minTokensFee` to any arbitrary value without enforcing a maximum limit.

```
function setMinTokensFee(uint256 _fee) external onlyOwner {
    minTokensFee = _fee;
}
```

This value is later converted into an `earlyFee` during settlement when a depositor completes a session before any proof submission:

```
if (completedBy == session.depositor && session.proofs.length == 0 && minTokensFee > 0) {
    earlyFee = (minTokensFee * session.pricePerToken) / PRICE_PRECISION;
    if (hostPayment >= session.deposit) {
        earlyFee = 0;
    } else if (earlyFee > session.deposit - hostPayment) {
        earlyFee = session.deposit - hostPayment;
    }
}
```

Although settlement logic caps `earlyFee` to the remaining session deposit, the absence of an explicit upper bound enables configuration that can effectively consume the full refundable balance for early cancellations. This creates a governance/configuration footgun and weakens predictability for users.

In addition, the owner can front-run a depositor's `completeSessionJob()` by calling `setMinTokensFee()` with a high value just before the completion transaction is mined. When the depositor completes with no proofs, the same settlement logic applies using the new, higher `minTokensFee`, so the depositor receives a smaller refund (or none) and the host receives a larger early fee—all without the depositor having agreed to that fee at session creation. This creates a governance/configuration footgun and weakens predictability for users.

Found in commit: `f614355`.

Assets:

- src/JobMarketplaceWithModelsUpgradeable.sol [https://github.com/Fabstir/fabstir-compute-contracts#]

Status: Accepted

Classification

Impact Rate: 3/5

Likelihood Rate: 2/5

Exploitability: Dependent

Complexity: Simple

Severity: Low

Recommendations

Remediation: Introduce a protocol-level maximum for `minTokensFee` and validate in `setMinTokensFee`, for example:

- cap by absolute token amount (`MAX_MIN_TOKENS_FEE`), and/or
- cap relative to `proofInterval` (e.g., `minTokensFee <= proofInterval`), and/or
- enforce a bounded economic ratio derived from pricing constraints.
- only allow `setMinTokensFee` to take effect after a delay (e.g. 24–48 hours).

Emit an event on updates for transparent monitoring of fee parameter changes.

Resolution: Partially fixed. An explicit upper bound (MAX_MIN_TOKENS_FEE = 10000) is now enforced in `setMinTokensFee()`, preventing arbitrarily large fee configuration. However, the front-running vector remains: the fee is still applied at settlement time rather than being locked at session creation, so the owner can still change it between a depositor's session creation and completion:

```
function setMinTokensFee(uint256 _fee) external onlyOwner {
  require(_fee <= MAX_MIN_TOKENS_FEE, "Fee too high");
  uint256 oldFee = minTokensFee;
  minTokensFee = _fee;
  emit MinTokensFeeUpdated(oldFee, _fee);
}
```

Revised commit: `df1f2e4`.

F-2026-15278 - Host Can Unregister During Active Sessions and Withdraw Stake While Continuing to Earn - Low

Description:

`unregisterNode()` allows a host to unregister and withdraw their full stake at any time. There is no check for whether the host has active sessions in the **JobMarketplace**. After unregistering, the host can still submit proofs and complete existing sessions because `submitProofOfWork()` and `completeSessionJob()` only verify `msg.sender == session.host`, not the host's registration or staking status. The host continues to earn from sessions with zero stake at risk and cannot be slashed for misbehavior.

`unregisterNode()` performs no session-awareness check:

```
function unregisterNode() external nonReentrant {
    require(nodes[msg.sender].operator != address(0), "Not registered");
    uint256 stakeToReturn = nodes[msg.sender].stakedAmount;
    bytes32[] memory models = nodes[msg.sender].supportedModels;
    for (uint i = 0; i < models.length; i++) {
        _removeNodeFromModel(models[i], msg.sender);
    }
    _removeFromActiveNodes(msg.sender);
    delete nodes[msg.sender];
    fabToken.safeTransfer(msg.sender, stakeToReturn);
    emit NodeUnregistered(msg.sender, stakeToReturn);
}
```

After this call, `nodes[host]` is deleted and the full stake is returned. However, `sessionJobs[jobId].host` in the **JobMarketplace** still holds the host's address. The proof and completion paths only check identity:

```
require(msg.sender == session.host, "Not host");
```

They do not query `nodeRegistry` for registration status. So the host can keep submitting proofs and completing sessions as normal. Meanwhile, `slashStake()` requires `nodes[host].operator != address(0)` and `nodes[host].active,` so it reverts for the unregistered host. There is no on-chain mechanism to penalize the host for misbehavior in those sessions.A host can exploit this by registering, accepting sessions from depositors, then calling `unregisterNode()` to recover their full stake before doing any work. The host then decides whether to serve the sessions honestly or not — either way, they earn from proven work with no stake at risk. If they produce garbage output or abandon sessions partway, the depositor has no slashing recourse.

The staking model is designed so that hosts have skin in the game: misbehavior can be punished by slashing their stake. A host who unregisters during active sessions withdraws their entire stake while retaining the ability to earn from those sessions. The protocol loses its only enforcement mechanism (slashing) over that host's active sessions. Depositors in those sessions bear all the risk — they have already deposited funds and cannot slash or otherwise penalize the host. This undermines the economic security model of the protocol.

Found in commit: `f614355`.

Assets:

- src/NodeRegistryWithModelsUpgradeable.sol [https://github.com/Fabstir/fabstir-compute-contracts#]

Status: Accepted

Classification

Impact Rate: 2/5

Likelihood Rate: 2/5

Exploitability: Semi-Dependent

Complexity: Simple

Severity: Low

Recommendations

Remediation: Prevent a host from unregistering while they have active sessions. This can be enforced by either:

- Tracking active session count per host in the **JobMarketplace** (increment on session creation, decrement on completion/timeout) and exposing a view function that `unregisterNode()` checks before allowing unregistration.
- Introducing a "cooldown" or "pending unregistration" state where the host signals intent to unregister, and actual stake withdrawal only happens after all active sessions are settled.
- At minimum, re-checking host registration and staking status in `submitProofOfWork()` so that an unregistered host cannot submit further proofs for existing sessions.

Resolution: The given finding is accepted, and no further changes were applied.

F-2026-15332 - Delegate Cannot Complete or Manage Sessions They Created - Low

Description:

The delegation system allows authorized delegates to create sessions on behalf of a payer via `createSessionForModelAsDelegate()`. However, once a session is created, the delegate has no ability to complete, cancel, or manage it. The or `completeSessionJob()` function restricts access to the depositor and host only:

```
function completeSessionJob(uint256 jobId, string calldata conversationCID) e
xternal nonReentrant {
   SessionJob storage session = sessionJobs[jobId];
   require(session.status == SessionStatus.Active, "Not active");
   require(
       msg.sender == session.depositor || msg.sender == session.host,
       "Not depositor or host"
   );
   ...
}
```

When a delegate creates a session, `_initializeSession` records the payer as the depositor, not the delegate:

```
_initializeSession(sessionId, params, payer);  // payer becomes session.depos
itor
```

The delegate's address is never stored in the `SessionJob` struct and is only emitted in the `SessionCreatedByDelegate` event. After creation, the delegate is completely detached from the session.

The stated purpose of the delegation system is to support Coinbase Smart Wallet sub-accounts and similar autonomous service accounts that operate on behalf of a payer. In practice, a delegate can open sessions but cannot close them. If the host becomes unresponsive and fails to submit proofs, the delegate cannot trigger an early cancellation to recover the payer's funds — the payer must personally intervene. This reduces delegation to a one-way "spend only" mechanism and limits its usefulness for autonomous or programmatic workflows where the delegate is expected to manage the full session lifecycle independently.

Found in commit : `df1f2e4`.

Assets:

- src/JobMarketplaceWithModelsUpgradeable.sol [https://github.com/Fabstir/fabstir-compute-contracts#]

Status: Accepted

Classification

Impact Rate: 3/5

Likelihood Rate: 3/5

Exploitability: Semi-Dependent

Complexity: Simple

Severity: Low

Recommendations

Remediation: Store the delegate's address in the `SessionJob` struct (or in a separate mapping) and allow the delegate to call `completeSessionJob()` with the same permissions as the depositor.

Resolution: The given finding is accepted, and no further changes were applied.

F-2026-15333 - Delegate Authorization State Can Be Stale and Misleading - Low

Description:

The delegate authorization system has two related inconsistencies that can produce misleading state:First, `isDelegateAuthorized()` only checks the active flag and ignores all other constraints in `DelegateConfig`:

```
function isDelegateAuthorized(address depositor, address delegate) external view returns (bool) {
   return delegateConfigs[depositor][delegate].active;
}
```

This returns true for delegates that have expired (`validUntil` has passed), exceeded their spending cap (`spent >= totalCap`), or are scoped to a host/model that no longer exists. Any off-chain system or UI relying on this function to display delegate status will show an authorized delegate when, in practice, every session creation attempt would revert.

Second, `authorizeDelegate()` only modifies `active` and `validUntil`, leaving all other config fields untouched:

```
function authorizeDelegate(address delegate, bool authorized) external {
   require(delegate != address(0), "Zero addr");
   require(delegate != msg.sender, "Self deleg");
   delegateConfigs[msg.sender][delegate].active = authorized;
   if (authorized) delegateConfigs[msg.sender][delegate].validUntil = 0;
   emit DelegateAuthorized(msg.sender, delegate, authorized);
}
```

If a payer previously called `configureDelegate()` with restrictions, then revoked the delegate via `authorizeDelegate(delegate, false)`, and later re-activated via `authorizeDelegate(delegate, true)`, the old restrictions silently persist. The payer may expect a clean re-authorization with unrestricted access, but the delegate inherits stale scope restrictions and a stale spent counter that could silently block session creation.

Third, the function selectively resets `validUntil` = 0 on re-activation to prevent a stale expiry from blocking the delegate, but does not apply the same logic to other fields that can equally block a re-activated delegate. A `spent` counter that already reached `totalCap` from a prior configuration will still block session creation even after re-activation, with no indication to the payer that the old cap is still in effect. The fix for stale expiry was applied in isolation without addressing the same class of problem in the remaining config fields.

Found in commit : `df1f2e4`.

Assets:

- src/JobMarketplaceWithModelsUpgradeable.sol [https://github.com/Fabstir/fabstir-compute-contracts#]

Status: Accepted

Classification

Impact Rate: 2/5

Likelihood Rate: 3/5

Exploitability: Independent

Complexity: Simple

Severity: Low

Recommendations

Remediation:

- Update `isDelegateAuthorized()` to validate all active constraints (expiry, spending cap) before returning true.
- Replace `authorizeDelegate()` with a `revokeDelegate()` function that deletes the entire `DelegateConfig` struct, eliminating all stale-state issues.

Resolution: The given finding is accepted, and no further changes were applied.

F-2026-15335 - Delegate Configuration Limited to Single Host/Model — No Multi-Config Support Per Delegate - Low

Description:

The delegate configuration mapping stores a single `DelegateConfig` per payer-delegate pair:

```
mapping(address => mapping(address => DelegateConfig)) public delegateConfigs
;
```

Each `DelegateConfig` supports only one `allowedHost` and one `allowedModel`. Since `configureDelegate()` overwrites the entire struct on every call, a payer cannot assign multiple host/model combinations to the same delegate address:

```
delegateConfigs[msg.sender][delegate] = DelegateConfig({
    maxPerSession: maxPerSession,
    totalCap: totalCap,
    spent: 0,
    validUntil: validUntil,
    active: true,
    allowedHost: allowedHost,
    allowedModel: allowedModel
});
```

If a payer calls `configureDelegate(delegate, ..., hostA, modelX)` followed by `configureDelegate(delegate, ..., hostB, modelY)`, the second call completely replaces the first. The delegate loses access to Host A / Model X and can only use Host B / Model Y. There is no way to accumulate multiple allowed host/model pairs for the same delegate.The enforcement is equally rigid — either the delegate is locked to exactly one host and one model, or must be granted unrestricted access via `address(0)` / `bytes32(0)`:

```
if (dc.allowedHost != address(0)) require(host == dc.allowedHost, "Wrong host
");
if (dc.allowedModel != bytes32(0)) require(modelId == dc.allowedModel, "Wrong
model");
```

The delegate scoping is effectively unusable for any real-world scenario where a delegate needs access to more than one host or model. A Coinbase Smart Wallet sub-account managing AI inference sessions would typically need to select from multiple available hosts running multiple models. The payer is forced to choose between overly restrictive access (one host, one model) or no restrictions at all, with no middle ground. Using separate delegate addresses per combination is impractical

as it fragments spending caps — each combination gets its own independent `totalCap` and spent counter — and requires managing multiple keys.

Found in commit : `df1f2e4`.

Assets:

- src/JobMarketplaceWithModelsUpgradeable.sol [https://github.com/Fabstir/fabstir-compute-contracts#]

Status: Accepted

Classification

Impact Rate: 2/5

Likelihood Rate: 3/5

Exploitability: Independent

Complexity: Simple

Severity: Low

Recommendations

Remediation: Change the single values to arrays in the struct and apply the required checks in `createSessionForModelAsDelegate()` function:

```
struct DelegateConfig {
    uint128 maxPerSession;
    uint128 totalCap;
    uint128 spent;
    uint64 validUntil;
    bool active;
    address[] allowedHosts;
    bytes32[] allowedModels;
}
```

Resolution: The Fabstir team stated that this design is intentional. The finding is accepted, and no further changes are applied.

F-2026-14873 - Code Duplication in createSessionFromDeposit() - Info

Description:

The `createSessionFromDeposit()` function duplicates validation and initialization logic that already exists in the helper functions `_validateSessionParams()` and `_initializeSession()`.
Other session creation functions (`createSessionJob`, `createSessionJobWithToken`, `createSessionJobForModel`, `createSessionJobForModelWithToken`) properly use these helpers, but `createSessionFromDeposit()` implements the same logic inline.

```
    function createSessionFromDeposit(
      address host,
      address paymentToken,
      uint256 deposit,
      uint256 pricePerToken,
      uint256 maxDuration,
      uint256 proofInterval
  ) external nonReentrant whenNotPaused returns (uint256 sessionId) {
      require(pricePerToken > 0, "Invalid price");
      require(maxDuration > 0 && maxDuration <= 365 days, "Invalid duration"
);
      require(proofInterval > 0, "Invalid proof interval");
      require(host != address(0), "Invalid host");

      // ... rest of the code

      sessionId = nextJobId++;
      SessionJob storage session = sessionJobs[sessionId];
      session.id = sessionId;
      session.depositor = msg.sender;
      session.host = host;
      session.paymentToken = paymentToken;
      session.deposit = deposit;
      session.pricePerToken = pricePerToken;
      session.maxDuration = maxDuration;
      session.startTime = block.timestamp;
      session.lastProofTime = block.timestamp;
      session.proofInterval = proofInterval;
      session.status = SessionStatus.Active;
      userSessions[msg.sender].push(sessionId);
      hostSessions[host].push(sessionId);

      // ... rest of the code
  }
```

Duplicated code increases contract size and deployment costs.

Assets:

- src/JobMarketplaceWithModelsUpgradeable.sol [https://github.com/Fabstir/fabstir-compute-contracts#]

Status: Accepted

Classification

Impact Rate: 1/5

Likelihood Rate: 5/5

Exploitability: Independent

Complexity: Simple

Severity: Info

Recommendations

Remediation: Refactor `createSessionFromDeposit()` to use the existing `_validateSessionParams()` and `_initializeSession()` helper functions, keeping only the balance-deduction logic that is unique to this function.

Resolution: Accepted in `1775cb2`:

Session-from-deposit flow was refactored to reuse shared helpers instead of inline duplication in `createSessionFromDepositForModel()` function. Therefore, the original finding is resolved; however, the newly introduced `createSessionForModelAsDelegate()` function inlines validation and initialization logic that is already encapsulated in `_validateSessionParams()` and `_initializeSession()`. This code duplication increases the risk of future inconsistencies if one path is updated without the other, and should be refactored to reuse the existing internal helpers

F-2026-14908 - Unused onlyRegisteredHost Modifier Contains Empty Implementation - Info

Description:

The `onlyRegisteredHost` modifier is defined in the contract but has an empty implementation and is never used by any function. The modifier body contains only a placeholder comment and the _; statement, providing no actual access control.

```
modifier onlyRegisteredHost(address host) {
  // Just check if host is registered by looking at operator
  // NodeRegistryWithModels has different return signature
  _;
}
```

Impact:

- The modifier is never applied to any function in the contract
- Contradicts the codebase's own validation pattern, causing confusion

Assets:

- src/JobMarketplaceWithModelsUpgradeable.sol [https://github.com/Fabstir/fabstir-compute-contracts#]

Status: Fixed

Classification

Impact Rate: 1/5

Likelihood Rate: 5/5

Exploitability: Independent

Complexity: Simple

Severity: Info

Recommendations

Remediation: Remove the unused modifier to improve code clarity and prevent potential misuse.

Resolution: Fixed in `fa8743c`:

> The unused `onlyRegisteredHost` modifier was removed from **JobMarketplaceWithModelsUpgradeable**.

F-2026-14916 - Pre-Deposited Users Cannot Create Model-Specific Sessions - Info

Description:

The contract provides two payment flows for creating sessions:

1. Direct payment: `createSessionJob()` and `createSessionJobForModel()` - users pay at creation time
2. Pre-deposit: `depositNative()` / `depositToken()` then `createSessionFromDeposit()` - users deposit funds first, then create sessions

However, users who pre-deposit funds have no way to create model-specific sessions. There is no `createSessionFromDepositForModel()` function equivalent to `createSessionJobForModel()`.

Impact:

- Users who prefer the pre-deposit flow cannot use model-specific features
- Users wanting model-specific sessions must switch to direct payment, losing the benefits of pre-depositing (gas efficiency for multiple sessions, simpler wallet management)

Assets:

- src/JobMarketplaceWithModelsUpgradeable.sol [https://github.com/Fabstir/fabstir-compute-contracts#]

Status: Fixed

Classification

Impact Rate: 1/5

Likelihood Rate: 4/5

Exploitability: Independent

Complexity: Simple

Severity: Info

Recommendations

Remediation: Add `createSessionFromDepositForModel()` to provide feature parity.

Resolution: Fixed in `3d366d8`:

JobMarketplaceWithModelsUpgradeable now includes `createSessionFromDepositForModel`, enabling model-specific session creation from pre-deposited balances.

F-2026-14967 - Blacklisted Proposer Permanently Blocks Model Execution and Re-Proposals - Info

Description:

When a proposal is approved, the `executeProposal()` function attempts to refund the 100 FAB proposal fee to the proposer. If the proposer has been blacklisted by the governance token (e.g., tokens with blacklist functionality like USDC-style implementations), the `safeTransfer` call will revert, causing the entire transaction to fail. Since the transaction reverts, `proposal.executed` remains false and the proposal stays in `activeProposals`. This prevents:

1. The approved model from being added to the registry
2. Any future re-proposals for the same `modelId`, as `_clearOldProposal()` requires the proposal to be executed before clearing

This effectively makes the `modelId` permanently unusable until it is added by the platform owner.

`executeProposal()` - transfer reverts if proposer blacklisted:

```
function executeProposal(bytes32 modelId) external {
  ModelProposal storage proposal = proposals[modelId];
  require(!proposal.executed, "Already executed");
  require(block.timestamp > proposal.endTime, "Voting still active");
  proposal.executed = true;  // Set before transfer
  // ...
  if (approved) {
      models[modelId] = proposal.modelData;
      // ...
  }
  if (approved) {
      // Reverts if proposer is blacklisted - entire tx fails
      governanceToken.safeTransfer(proposal.proposer, PROPOSAL_FEE);
  }
  _removeFromActiveProposals(modelId);
  // ...
}
```

Assets:

- src/ModelRegistryUpgradeable.sol [https://github.com/Fabstir/fabstir-compute-contracts#]

Status: Accepted

Classification

Impact Rate: 1/5

Likelihood Rate: 2/5

Exploitability: Independent

Complexity: Simple

Severity: Info

Recommendations

Remediation: Use pull-over-push refund mechanism to prevent transfer failures from blocking execution:

```
mapping(address => uint256) public pendingRefunds;
if (approved) {
   pendingRefunds[proposal.proposer] += PROPOSAL_FEE;
}
function claimRefund() external {
   uint256 amount = pendingRefunds[msg.sender];
   require(amount > 0, "No refund");
   pendingRefunds[msg.sender] = 0;
   governanceToken.safeTransfer(msg.sender, amount);
}
```

Resolution: Accepted:

The issue remains open and is explicitly scheduled for a future upgrade cycle.

Client comment confirms planned migration of governance proposal-fee refunds to a pull-payment pattern, with additional governance-specific storage required before implementation.

Current execution path can still be blocked by refund transfer failure during `executeProposal`.

F-2026-14976 - Host Signature In submitProofOfWork() Adds No Security Value - Info

Description:

`submitProofOfWork()` function requires the caller to be the session host and to pass a proof that `ProofSystemUpgradeable.verifyAndMarkComplete()` accepts. The contract checks that the signer is the host and that proof hash has not been used before.

```
    function submitProofOfWork(…) external nonReentrant whenNotPaused {
      //…
      if (address(proofSystem) != address(0)) {
          // Construct 97-byte proof: proofHash (32) + signature (65)
          bytes memory proof = abi.encodePacked(proofHash, signature);
          require(
              proofSystem.verifyAndMarkComplete(proof, msg.sender, tokensCla
imed),
              "Invalid proof signature"
          );
          verified = true;
      }
      //…
  }
```

The host is the one who chooses `proofHash` and `tokensClaimed`, creates the signature, and submits the transaction. So the signer and the beneficiary are the same, and the signed data is entirely under the host's control.

```
    function verifyAndMarkComplete(bytes calldata proof, address prover, uint
256 claimedTokens)
      external
      returns (bool)
  {
      // First verify using internal function
      if (!_verifyHostSignature(proof, prover, claimedTokens)) {
          return false;
      }
      // Extract proof hash and mark as verified
      bytes32 proofHash;
      assembly {
          proofHash := calldataload(proof.offset)
      }
      verifiedProofs[proofHash] = true;
      emit ProofVerified(proofHash, prover, claimedTokens);
```

```
        return true;
    }
```

A host can therefore pick any proof hash, claim any allowed token amount, sign it, and submit; `verifyAndMarkComplete()` passes but adds no guarantee beyond the existing requirement that the caller is the host. For a signature to add security it would need to come from another party or to commit to data the signer cannot forge; neither holds here. The host signature in `submitProofOfWork()` and the verification in `ProofSystemUpgradeable` thus add complexity and gas without adding meaningful security over the `msg.sender == session.host` check and the contract's rate and deposit limits.

Assets:

- src/JobMarketplaceWithModelsUpgradeable.sol [https://github.com/Fabstir/fabstir-compute-contracts#]
- src/ProofSystemUpgradeable.sol [https://github.com/Fabstir/fabstir-compute-contracts#]

Status: Fixed

Classification

Impact Rate: 1/5

Likelihood Rate: 4/5

Exploitability: Independent

Complexity: Simple

Severity: Info

Recommendations

Remediation: If no additional guarantee is desired, consider removing the signature parameter and proof system verification from this path and relying on the host check and economic limits. If attestation by another party or binding to off-chain proof data is desired, change the design so that either a different party signs or the signed payload includes data the host cannot forge.

Resolution: Fixed in `aee7251`:

`submitProofOfWork` no longer accepts a signature parameter and no longer performs host-signature verification as a gating step.

```
function markProofUsed(
    bytes32 proofHash,
```

```
    address prover,
    uint256 claimedTokens,
    bytes32 modelId
) external override returns (bool) {
    require(authorizedCallers[msg.sender] || msg.sender == owner(), "Unauthor
ized");

    // Replay protection - return false if already used
    if (verifiedProofs[proofHash]) return false;

    verifiedProofs[proofHash] = true;
    emit ProofVerified(proofHash, prover, claimedTokens);

    return true;
}
```

The proof flow was simplified to host authorization (`msg.sender == session.host`) plus replay protection through proof-hash consumption (`markProofUsed`) and economic/session limits.

F-2026-15002 - Unused Functions In ProofSystemUpgradeable - Info

Description:

Several functions in `ProofSystemUpgradeable` are never called by any contract in the codebase. The only integration point is JobMarketplace, which uses `verifyAndMarkComplete()`. The rest of the proof and circuit API is unused and effectively dead code, which increases complexity, maintenance cost, and attack surface without providing current benefit.

The following are not invoked by any contract:

- **verifyBatch** — Batch verification and marking of proofs; no caller in production.
- **estimateBatchGas** — Gas estimation for batch verification; no caller.
- **recordVerifiedProof** — Mark a proof hash as verified without a signature; only used in tests; JobMarketplace does not call it.
- **registerModelCircuit** — Register a model–circuit mapping; no caller.
- **isCircuitRegistered** — Check if a circuit is registered; no caller.
- **getModelCircuit** — Get circuit hash for a model; no caller.

Assets:

- src/ProofSystemUpgradeable.sol [https://github.com/Fabstir/fabstir-compute-contracts#]

Status: Fixed

Classification

Impact Rate: 1/5

Likelihood Rate: 5/5

Exploitability: Independent

Complexity: Simple

Severity: Info

Recommendations

Remediation: Either remove the unused functions or document the planned use and implement the necessary functionalities.

Resolution: Fixed in `0606d07`:

Unused **ProofSystem** API surface was removed during contract simplification.

F-2026-15003 - Missing Zero-Address Checks Across Contracts - Info

Description: Several address parameters and storage values are used in critical paths without a prior check that they are not the zero address. That can lead to failed transfers, disabled features, or inconsistent state when configuration is wrong or incomplete.

- **NodeRegistryWithModelsUpgradeable:** `slashStake()` does not check that treasury is not address(0) before transferring slashed tokens.
- **JobMarketplaceWithModelsUpgradeable:** `setProofSystem()` does not check that `_proofSystem` is not address(0).
- **JobMarketplaceWithModelsUpgradeable:** `initializeChainConfig()` does not check that `ChainConfig` fields are not address(0) before storing the config.

Assets:

- src/JobMarketplaceWithModelsUpgradeable.sol [https://github.com/Fabstir/fabstir-compute-contracts#]
- src/NodeRegistryWithModelsUpgradeable.sol [https://github.com/Fabstir/fabstir-compute-contracts#]

Status: Accepted

Classification

Impact Rate: 3/5

Likelihood Rate: 1/5

Exploitability: Independent

Complexity: Simple

Severity: Info

Recommendations

Remediation: Implement required zero address checks for the given functions.

Resolution: Accepted in `d366305`:

Zero-address validation was added for **JobMarketplaceWithModelsUpgradeable**.`setProofSystem`, preventing

accidental disabling of proof verification wiring through null configuration. The issue remains partially unresolved in other paths:

- **JobMarketplaceWithModelsUpgradeable**. `initializeChainConfig` still does not validate non-zero `ChainConfig` address fields before persisting.
- **NodeRegistryWithModelsUpgradeable**. `slashStake` still does not enforce a local non-zero treasury check at transfer time.

F-2026-15065 - Unfinalized Circuits Functionality in ProofSystemUpgradeable Contract - Info

Description:

The `ProofSystemUpgradeable` contract introduces `registeredCircuits` and `modelCircuits` mappings along with `registerModelCircuit`, `isCircuitRegistered`, `getModelCircuit` functions.

The functions are not properly documented and are not used by the other system contracts. The state variables do not serve for any kind of proving process the contract is focused on.

Additionally, the function processes `model` as an `address` type, which is semantically incorrect, as models are neither smart contracts nor EOAs on the blockchain.

```
function registerModelCircuit(address model, bytes32 circuitHash)
function getModelCircuit(address model)
```

The inconsistency does not point a direct vulnerability, it outlines a redundant functionality trying to mislead auditors and developers about the overall system features, particularly, it introduces model circuits feature in a wrong place which is not properly integrated in the total system workflow.

Assets:

- src/ProofSystemUpgradeable.sol [https://github.com/Fabstir/fabstir-compute-contracts#]

Status: Fixed

Classification

Impact Rate: 1/5

Likelihood Rate: 5/5

Exploitability: Independent

Complexity: Simple

Severity: Info

Recommendations

Remediation: Consider removing the redundant endpoints and implementing the feature with a proper integration in the right place, e.g. in the `ModelRegistryUpgradeable` contract.

Resolution: Fixed in `0606d07`:

Circuit-related storage and endpoints were removed from **ProofSystemUpgradeable**.

`registeredCircuits`, `modelCircuits`, and associated functions (`registerModelCircuit`, `isCircuitRegistered`, `getModelCircuit`) are no longer part of the active implementation.

F-2026-15066 - Redundant Function Wrapper in ProofSystemUpgradeable Contract - Info

Description:

The `ProofSystemUpgradeable` contract introduces the `_verifyHostSignatureInternal` function. The function points itself as a helper for batch signature verification.

However, the function is redundant. It only wraps `_verifyHostSignature` and adds no value.

```
function _verifyHostSignatureInternal(bytes calldata proof, address prover, u
int256 claimedTokens)
    internal
    view
    returns (bool)
{
    return _verifyHostSignature(proof, prover, claimedTokens);
}
```

The redundancy increases code complexity.

Assets:

- src/ProofSystemUpgradeable.sol [https://github.com/Fabstir/fabstir-compute-contracts#]

Status: Fixed

Classification

Impact Rate: 1/5

Likelihood Rate: 5/5

Exploitability: Independent

Complexity: Simple

Severity: Info

Recommendations

Remediation: Consider directly using the `_verifyHostSignature` function instead of the redundant wrapping, remove the `_verifyHostSignatureInternal` function.

Resolution: Fixed in `0606d07`:

`_verifyHostSignatureInternal` is no longer present in **ProofSystemUpgradeable**.

Legacy signature-verification helpers and related dead verification paths were removed during **ProofSystem** simplification, eliminating the redundant wrapper and associated complexity.

F-2026-15067 - Inefficient Revert Messages - Info

Description: Using revert string messages increases the size of a smart contract and its deployment costs.

Since Solidity version `0.8.4` custom errors are available. Custom Errors provide a way for developers to define descriptive and semantically meaningful error conditions without relying on string messages.

The revert strings are used in the `HostEarningsUpgradeable`, `JobMarketplaceWithModelsUpgradeable`, `ModelRegistryUpgradeable`, `NodeRegistryWithModelsUpgradeable`, `ProofSystemUpgradeable` contracts.

Assets:

- src/JobMarketplaceWithModelsUpgradeable.sol [https://github.com/Fabstir/fabstir-compute-contracts#]
- src/NodeRegistryWithModelsUpgradeable.sol [https://github.com/Fabstir/fabstir-compute-contracts#]
- src/ModelRegistryUpgradeable.sol [https://github.com/Fabstir/fabstir-compute-contracts#]
- src/ProofSystemUpgradeable.sol [https://github.com/Fabstir/fabstir-compute-contracts#]
- src/HostEarningsUpgradeable.sol [https://github.com/Fabstir/fabstir-compute-contracts#]

Status: Mitigated

Classification

Impact Rate: 1/5

Likelihood Rate: 3/5

Exploitability: Independent

Complexity: Simple

Severity: Info

Recommendations

Remediation: To reduce gas costs during contract deployment, consider using custom errors instead of revert strings.

Solidity compiler supports the use of Custom Errors within require statements since version `0.8.26` for the IR pipeline and since version

`0.8.27` for the legacy pipeline. For the older Solidity versions, the pattern `if (!condition) revert CustomError()` should be used.

Resolution: Mitigated in `6032423`:

Revert-string footprint was reduced through shorter require messages in **JobMarketplaceWithModelsUpgradeable**, lowering bytecode pressure.

Custom errors were not adopted across the referenced contracts, and string-based require checks remain widely used in **HostEarningsUpgradeable**, **JobMarketplaceWithModelsUpgradeable**, **ModelRegistryUpgradeable**, **NodeRegistryWithModelsUpgradeable**, and **ProofSystemUpgradeable**.

Deployment gas overhead from revert strings is therefore reduced but not fully eliminated.

F-2026-15279 - verified Field in ProofSubmission Is Hardcoded to true — Dead Storage - Info

Description:

The **ProofSubmission** struct contains a verified field of type bool, and its NatSpec describes it as "Whether the proof was cryptographically verified." However, the field is always set to true at the only location where ProofSubmission is created — inside `submitProofOfWork()`:

```
        bool verified = true;
        // S5: Store proof hash and CID instead of full proof
        session.lastProofHash = proofHash;
        session.lastProofCID = proofCID;
        // Store proof submission with verification status
        session.proofs.push(
            ProofSubmission({
                proofHash: proofHash,
                tokensClaimed: tokensClaimed,
                timestamp: block.timestamp,
                verified: verified,
                deltaCID: deltaCID
            })
        );
```

No code path ever sets verified to false, and no logic in the contract reads or branches on its value. The field is only surfaced through the `getProofSubmission()` view function, where off-chain consumers may incorrectly interpret verified = true as proof that real cryptographic verification occurred on-chain. In practice, no such verification happens — the contract only checks replay protection via `markProofUsed()`.

Every call to `submitProofOfWork()` pays an extra storage write for a bool that never varies. Over many proof submissions across many sessions, this is wasted gas. More importantly, the field and its NatSpec ("Whether the proof was cryptographically verified") are misleading — off-chain integrators or auditing tools reading `getProofSubmission()` may assume each proof was individually verified on-chain, when in fact it was not. This creates a false sense of security.

```
    function getProofSubmission(uint256 sessionId, uint256 proofIndex)
      external
      view
      returns (bytes32 proofHash, uint256 tokensClaimed, uint256 timestamp,
bool verified, string memory deltaCID)
   {
      SessionJob storage session = sessionJobs[sessionId];
```

```
        require(proofIndex < session.proofs.length, "Invalid proof index");
        ProofSubmission storage proof = session.proofs[proofIndex];
        return (proof.proofHash, proof.tokensClaimed, proof.timestamp, proof.v
erified, proof.deltaCID);
    }
```

Found in commit: `f614355`.

Assets:

- src/JobMarketplaceWithModelsUpgradeable.sol [https://github.com/Fabstir/fabstir-compute-contracts#]

Status: Fixed

Classification

Impact Rate: 1/5

Likelihood Rate: 5/5

Exploitability: Independent

Complexity: Simple

The `verified` memory variable is removed and the `verified` field is now annotated as deprecated and retained only for storage layout compatibility.

Severity: Info

Recommendations

Remediation: Remove the `verified` field from the **ProofSubmission** struct entirely, or repurpose it when on-chain verification is implemented. If the field is retained as a placeholder for future functionality, update the NatSpec to clearly state it is not currently meaningful and always returns true.

F-2026-15330 - Redundant usdcAddress State Variable and Setter Introduce Unnecessary Complexity - Info

Description:

The `JobMarketplaceWithModelsUpgradeable` contract maintains a dedicated `usdcAddress` state variable, a hardcoded USDC address in `initialize()`, two USDC-specific constants (`USDC_MIN_DEPOSIT`, `USDC_MAX_DEPOSIT`), and a dedicated `setUsdcAddress()` setter. However, USDC receives no special treatment anywhere in the contract's business logic — sessions, deposits, withdrawals, and settlements all use the generic `acceptedTokens` / `tokenMinDeposits` / `tokenMaxDeposits` system regardless of whether the token is USDC or any other ERC20.

```
usdcAddress = 0x036CbD53842c5426634e7929541eC2318f3dCF7e; // hardcoded Base S
epolia
acceptedTokens[usdcAddress] = true;
tokenMinDeposits[usdcAddress] = USDC_MIN_DEPOSIT;
tokenMaxDeposits[usdcAddress] = USDC_MAX_DEPOSIT;
```

Everything the `usdcAddress` variable provides can be achieved by a single `addAcceptedToken()` call in a post-deployment script, without the dedicated storage slot, the hardcoded testnet address in production code, or the deactivation side effect.

```
    function addAcceptedToken(address token, uint256 minDeposit, uint256 maxD
eposit) external {
        require(msg.sender == treasuryAddress || msg.sender == owner(), "Not a
dmin");
        require(!acceptedTokens[token], "Already set");
        require(minDeposit > 0, "Bad min");
        require(maxDeposit > minDeposit, "Max < min");
        require(token != address(0), "Zero addr");
        acceptedTokens[token] = true;
        tokenMinDeposits[token] = minDeposit;
        tokenMaxDeposits[token] = maxDeposit;
        emit TokenAccepted(token, minDeposit, maxDeposit);
    }
```

The extra state variable and constants add storage cost and audit surface for functionality that is fully redundant with the existing generic token acceptance system.

Found in: `df1f2e4`.

Assets:

- src/JobMarketplaceWithModelsUpgradeable.sol [https://github.com/Fabstir/fabstir-compute-contracts#]

Status: Accepted

Classification

Impact Rate: 1/5

Likelihood Rate: 5/5

Exploitability: Independent

Complexity: Simple

Severity: Info

Recommendations

Remediation: Remove the `usdcAddress` state variable, `USDC_MIN_DEPOSIT`, `USDC_MAX_DEPOSIT` constants, and `setUsdcAddress()` function. Configure the initial accepted token via `addAcceptedToken()` in the deployment script or through a post-initialization admin call. If backward compatibility.

Resolution: The given finding is accepted, and no further changes are applied.

F-2026-15336 - No Function to Remove or Disable an Accepted Token - Info

Description:

The contract provides `addAcceptedToken()` to whitelist new ERC20 tokens but has no corresponding function to remove or disable them. Once a token is added, `acceptedTokens[token]` is permanently true:

```
function addAcceptedToken(address token, uint256 minDeposit, uint256 maxDepos
it) external {
   require(msg.sender == treasuryAddress || msg.sender == owner(), "Not admin
");
   require(!acceptedTokens[token], "Already set");
   require(minDeposit > 0, "Bad min");
   require(maxDeposit > minDeposit, "Max < min");
   require(token != address(0), "Zero addr");
   acceptedTokens[token] = true;
   tokenMinDeposits[token] = minDeposit;
   tokenMaxDeposits[token] = maxDeposit;
   emit TokenAccepted(token, minDeposit, maxDeposit);
}
```

There is no `removeAcceptedToken()` function, and the existing admin functions cannot disable a token indirectly: `updateTokenMinDeposit()` requires `minDeposit > 0`, `updateTokenMaxDeposit()` requires `maxDeposit > tokenMinDeposits[token]`, and `acceptedTokens[token]` has no direct setter. The only mechanism that sets `acceptedTokens[token] = false` is `setUsdcAddress()`, which only affects the currently stored `usdcAddress`. Additionally, `addAcceptedToken()` prevents re-adding an already accepted token (`require(!acceptedTokens[token])`), so if a token is added with incorrect configuration, it cannot be removed and re-added — the admin must correct the values through separate `updateTokenMinDeposit()` / `updateTokenMaxDeposit()` calls.

Found in commit : `df1f2e4`.

Assets:

- src/JobMarketplaceWithModelsUpgradeable.sol [https://github.com/Fabstir/fabstir-compute-contracts#]

Status: Accepted

Classification

Impact Rate: 1/5

Likelihood Rate: 4/5

Exploitability: Dependent

Complexity: Simple

Severity: Info

Recommendations

Remediation: Add a `removeAcceptedToken()` function that disables the token for new deposits and session creation while preserving existing session functionality.

```
function removeAcceptedToken(address token) external {
    require(msg.sender == treasuryAddress || msg.sender == owner(), "Not admi
n");
    require(acceptedTokens[token], "Not accepted");

    acceptedTokens[token] = false;
    delete tokenMinDeposits[token];
    delete tokenMaxDeposits[token];

    emit TokenRemoved(token);
}
```

Resolution: The given finding is accepted, and no further changes are applied.

Disclaimers

Hacken Disclaimer

The smart contracts given for audit have been analyzed based on best industry practices at the time of the writing of this report, with cybersecurity vulnerabilities and issues in smart contract source code, the details of which are disclosed in this report (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

The report contains no statements or warranties on the identification of all vulnerabilities and security of the code. The report covers the code submitted and reviewed, so it may not be relevant after any modifications. Do not consider this report as a final and sufficient assessment regarding the utility and safety of the code, bug-free status, or any other contract statements.

While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only — we recommend proceeding with several independent audits and a public bug bounty program to ensure the security of smart contracts.

English is the original language of the report. The Consultant is not responsible for the correctness of the translated versions.

As part of Hacken's ongoing quality assurance process, we may conduct re-audits of select projects. These re-audits are performed independently from the original audit and are intended solely for internal quality control and improvement. Updated reports resulting from such re-audits will be shared privately with the respective clients and may be published on the Hacken website only with their explicit consent.
The sole authoritative source for finalized and most up-to-date versions of all reports remains the Audits section at https://hacken.io/audits/.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have vulnerabilities that can lead to hacks. Thus, the Consultant cannot guarantee the explicit security of the audited smart contracts.

Appendix 1. Definitions

Severities

When auditing smart contracts, Hacken is using a risk-based approach that considers **Likelihood**, **Impact**, **Exploitability** and **Complexity** metrics to evaluate findings and score severities.

Reference on how risk scoring is done is available through the repository in our Github organization:

hknio/severity-formula

Severity	Description
Critical	Critical vulnerabilities are usually straightforward to exploit and can lead to the loss of user funds or contract state manipulation.
High	High vulnerabilities are usually harder to exploit, requiring specific conditions, or have a more limited scope, but can still lead to the loss of user funds or contract state manipulation.
Medium	Medium vulnerabilities are usually limited to state manipulations and, in most cases, cannot lead to asset loss. Contradictions and requirements violations. Major deviations from best practices are also in this category.
Low	Major deviations from best practices or major Gas inefficiency. These issues will not have a significant impact on code execution.

Potential Risks

The "Potential Risks" section identifies issues that are not direct security vulnerabilities but could still affect the project's performance, reliability, or user trust. These risks arise from design choices, architectural decisions, or operational practices that, while not immediately exploitable, may lead to problems under certain conditions. Additionally, potential risks can impact the quality of the audit itself, as they may involve external factors or components beyond the scope of the audit, leading to incomplete assessments or oversight of key areas. This section aims to provide a broader perspective on factors that could affect the project's long-term security, functionality, and the comprehensiveness of the audit findings.

Appendix 2. Scope

The scope of the project includes the following smart contracts from the provided repository:

Scope Details	
Repository	https://github.com/Fabstir/fabstir-compute-contracts#
Commit	8edc68be530f53249c6845fb0c1fbdf9dec68e62
Retest 1	f614355dbc0fd346dff331a587fc24413a9f0664
Retest 2	df1f2e4d7725a98d76b821ee759cd2c59f9dfa21
Final commit	fba54b2bf1181dacc7ec8db43e0e16854df0483d
Whitepaper	N/A
Requirements	https://github.com/Fabstir/fabstir-compute-contracts/tree/main/docs
Technical Requirements	https://github.com/Fabstir/fabstir-compute-contracts/tree/main/docs

Asset	Type
src/HostEarningsUpgradeable.sol [https://github.com/Fabstir/fabstir-compute-contracts#]	Smart Contract
src/interfaces/UserOperation.sol [https://github.com/Fabstir/fabstir-compute-contracts#]	Smart Contract
src/JobMarketplaceWithModelsUpgradeable.sol [https://github.com/Fabstir/fabstir-compute-contracts#]	Smart Contract
src/ModelRegistryUpgradeable.sol [https://github.com/Fabstir/fabstir-compute-contracts#]	Smart Contract
src/NodeRegistryWithModelsUpgradeable.sol [https://github.com/Fabstir/fabstir-compute-contracts#]	Smart Contract
src/ProofSystemUpgradeable.sol [https://github.com/Fabstir/fabstir-compute-contracts#]	Smart Contract
src/utils/ReentrancyGuardUpgradeable.sol. [https://github.com/Fabstir/fabstir-compute-contracts#]	Smart Contract

Appendix 3. Additional Valuables

Additional Recommendations

The smart contracts in the scope of this audit could benefit from the introduction of automatic emergency actions for critical activities, such as unauthorized operations like ownership changes or proxy upgrades, as well as unexpected fund manipulations, including large withdrawals or minting events. Adding such mechanisms would enable the protocol to react automatically to unusual activity, ensuring that the contract remains secure and functions as intended.

To improve functionality, these emergency actions could be designed to trigger under specific conditions, such as:

- Detecting changes to ownership or critical permissions.
- Monitoring large or unexpected transactions and minting events.
- Pausing operations when irregularities are identified.

These enhancements would provide an added layer of security, making the contract more robust and better equipped to handle unexpected situations while maintaining smooth operations.

Frameworks and Methodologies

This security assessment was conducted in alignment with recognised penetration testing standards, methodologies and guidelines, including the NIST SP 800-115 – Technical Guide to Information Security Testing and Assessment, and the Penetration Testing Execution Standard (PTES), These assets provide a structured foundation for planning, executing, and documenting technical evaluations such as vulnerability assessments, exploitation activities, and security code reviews. Hacken's internal penetration testing methodology extends these principles to Web2 and Web3 environments to ensure consistency, repeatability, and verifiable outcomes.